Università degli Studi di Napoli
Federico II

A.A 2024-2025

# Manuale Tecnico DietiEstates25

# Indice


1 Documento dei Requisiti Software 3
1.1 Glossario 3
1.2 Modellazione target utenti 3
1.2.1 Utente 4
1.2.2 Agente immobiliare 4
1.2.3 Gestore di una agenzia immobiliare 5
1.3 Casi d'Uso e descrizione dei requisiti 5
1.3.1 Descrizione dei requisiti 6
1.4 Fully dressed use case diagrams 7
1.4.1 Ricerca di un immobile 7
1.4.2 Caricamento di un annuncio immobiliare 10
1.4.3 Aggiunta di un agente immobiliare 13
1.4.4 Promozione di un agente immobiliare 15

2 Architettura e Design del Sistema 17
2.1 Architettura del Sistema 17
2.1.1 Livello di Presentazione (Presentation Tier) 17
2.1.2 Livello di Business (Application Tier) 17
2.1.3 Livello Dati (Data Tier) 17
2.2 Tecnologie Adottate 18
2.2.1 Backend 18
2.2.2 Approfondimento: Sicurezza con Spring Security e OpenID Connect 18
2.2.3 Frontend 19
2.3 Design dell'Interfaccia Utente (UI) 20
2.4 Descrizione dello schema per la persistenza dei dati 20
2.4.1 Panoramica e vincoli generali 20
2.4.2 Tabelle principali 20
2.4.3 Funzioni e logica lato database 21
2.4.4 Motivazione per l'adozione di PostGIS 21
2.4.5 Uso di PostGIS nelle query di ricerca per raggio 21
2.4.6 La gestione delle immagini 22
2.5 Diagramma delle classi di design 23
2.5.1 Sommario dei package e classi principali 24
2.6 Diagrammi di sequenza 25
2.6.1 Creazione annuncio immobiliare 26
2.6.2 Ricerca annuncio immobiliare 26
2.6.3 Creazione agente immobiliare 27
2.6.4 Promozione agente immobiliare 28

3 Testing e valutazione dell'usabilità 29
3.1 Unit testing 29
3.1.1 Il metodo `updatePassword` 29
3.1.2 Il metodo `createPoint` 30
3.1.3 Il metodo `saveImage` 31
3.1.4 Il metodo `createListing` 31
3.2 Tecniche di valutazione dell'usabilità 32
3.2.1 Expert reviews 32
3.2.2 Esperimenti con utenti reali 32

# Documento dei Requisiti Software

## 1.1 Glossario

In questo capitolo vengono definiti i termini tecnici e i concetti chiave utilizzati all'interno della documentazione del progetto DietiEstates25. Lo scopo è fornire un riferimento univoco per facilitare la comprensione del dominio applicativo e delle funzionalità del sistema da parte di tutti gli stakeholder.

| Termine | Definizione |
|---|---|
| `Agente immobiliare` | Un professionista che lavora per conto di una agenzia immobiliare e che gestisce annunci immobiliari. |
| `Amministratore` | Utente con privilegi elevati all'interno del gestionale della propria agenzia. |
| `Cliente` | Un utente finale che utilizza il sistema per cercare annunci immobiliari. |
| `Immobile` | Proprietà gestita dagli agenti immobiliari all'interno del sistema DietiEstates25. |
| `Inserzione` | Annuncio di vendita o affitto di un immobile. |
| `Mappa interattiva` | Strumento che permette la localizzazione e ricerca degli immobili. |
| `Notifica` | Un messaggio inviato agli utenti (clienti o agenti immobiliari) per informarli di eventi significativi, come la prenotazione di una visita o il rifiuto di un appuntamento. Le notifiche vengono inviate via e-mail. |
| `Ricerca avanzata` | Funzionalità che consente agli utenti di filtrare gli immobili per criteri multipli. |

Tabella 1.1: Glossario dei termini

## 1.2 Modellazione target utenti

Nel contesto della gestione immobiliare, interagiscono tre figure chiave, ognuna con esigenze e obiettivi specifici:

- **Utente:** una persona interessata a trovare la casa ideale.
- **Agente immobiliare:** un professionista focalizzato sulla vendita e sulla gestione degli immobili.
- **Gestore agenzia:** una figura manageriale che coordina l'attività di un'agenzia immobiliare.

### 1.2.1 Utente

| | |
|---|---|
| Nome | Lucia |
| Cognome | Moretti |
| Età | 25 anni |
| Stato civile | Single |
| Titolo di Studio | Laurea Magistrale in Informatica |
| Professione | App developer |
| Tratti personali | Calma, paziente, e simpatica |

**Biografia** Lucia Moretti, nata a Napoli il 12 aprile del 1999, lavora in Apple come UX/UI developer. Vive in periferia, ma il lungo tragitto per raggiungere l'ufficio l'ha spinta a cercare una casa più vicina al lavoro. Vorrebbe un appartamento moderno, ben collegato e con una buona efficienza energetica, per ridurre i costi.

**Obiettivi** Lucia desidera avere un appartamento in affitto vicino alla sede di lavoro. Cerca una buona piattaforma per trovare una casa fatta su misura delle sue esigenze senza dover non andare in un agenzia immobiliare tradizionale. Prima di fissare un appuntamento vuole avere una panoramica di tutti gli immobili che possano soddisfare le sue esigenze da programmatrice.

**Interessi** Lucia ama immergersi nella scrittura e nella lettura, e per lei la casa dei sogni è un luogo che ispira storie, con una biblioteca accogliente e un angolo perfetto per scrivere. Essendo una persona socievole, immagina una casa ideale con spazi ampi e accoglienti per ospitare amici e organizzare cene ed eventi, dove ogni ambiente trasmetta calore e convivialità. La sua passione per la tecnologia la porta a desiderare una casa intelligente, dotata di dispositivi innovativi e soluzioni smart che rendano la quotidianità più semplice ed efficiente, permettendole di dedicarsi alle sue passioni senza pensieri.

**Frustrazioni e difficoltà** Ogni giorno, Lucia si ritrova a scorrere decine di annunci online con foto ingannevoli e descrizioni vaghe, solo per scoprire che molte case sono già state vendute o affittate. Quando finalmente trova qualcosa di interessante, contattare proprietari o agenzie diventa una sfida: risposte che non arrivano, siti che chiedono sottoscrizioni ad abbonamenti, richieste economiche fuori dal suo budget.

### 1.2.2 Agente immobiliare

| | |
|---|---|
| Nome | Andrea |
| Cognome | Rinaldi |
| Età | 45 anni |
| Titolo di studio | Geometra |
| Professione | Agente immobiliare |
| Stato civile | Sposato |
| Tratti personali | Disponibile, introverso e riflessivo |

**Biografia** Nasce a Milano nel 28 Agosto del 1980. È un agente immobiliare con esperienza e una formazione da Geometra. È sposato e gestisce anche un'azienda agricola di famiglia. Sebbene apprezzi l'ordine e la sicurezza, trova frustrante la gestione degli appuntamenti e delle disdette dei clienti, preferirebbe un sistema più efficiente per organizzare il suo lavoro.

**Obiettivi** Andrea è un agente immobiliare dinamico e organizzato, sempre alla ricerca di strumenti che possano ottimizzare il suo lavoro e migliorare l'esperienza dei clienti. Il suo obiettivo principale è gestire in modo efficiente il suo portafoglio di immobili, inserendo facilmente dettagli fondamentali come numero di stanze, piano, presenza di ascensore, classe energetica e servizi disponibili.

Per offrire una presentazione completa e coinvolgente, vuole poter visualizzare in un'unica piattaforma foto, video e informazioni dettagliate su ogni proprietà, comprese descrizione, prezzo e dimensioni. Inoltre, per garantire un servizio impeccabile, desidera ricevere notifiche automatiche sulle prenotazioni delle visite, così da organizzare al meglio il proprio tempo e offrire un supporto puntuale ai clienti.

**Frustrazioni/Difficoltà** Andrea si infastidisce quando i clienti cambiano o disdicono continuamente gli appuntamenti, chiamandolo ripetutamente in ufficio. Durante i periodi di maggiore afflusso, poi, diventa ancora più difficile gestire tutto con un'agenda cartacea. I continui slittamenti o cambiamenti degli appuntamenti gli fanno perdere facilmente traccia delle prenotazioni e degli orari disponibili, creando confusione e aumentando lo stress.

**Interessi** Andrea non è solo un agente immobiliare determinato, ma anche un imprenditore che gestisce con dedizione un'azienda di famiglia. La sua esperienza nel settore agricolo lo ha reso estremamente attento alle necessità dei clienti, motivo per cui cerca strumenti digitali che gli permettano di mantenere un'organizzazione impeccabile anche nel suo lavoro immobiliare.

Per lui, un'app che gli consenta di inserire facilmente tutti i dettagli degli immobili e monitorare ogni fase della vendita è essenziale per garantire efficienza e precisione. Sempre orientato al raggiungimento dei migliori risultati, Andrea trova particolarmente utile poter stare in stretto contatto con i propri clienti, così da non perdere nessuna opportunità e dimostrare la sua superiorità nel mercato immobiliare.

### 1.2.3 Gestore di una agenzia immobiliare

| | |
|---|---|
| Nome | Marco |
| Cognome | De Santis |
| Età | 55 anni |
| Stato civile | Sposato |
| Titolo di studio | Diploma di Liceo Classico |
| Professione | Gestore di un'agenzia immobiliare |
| Tratti personali | Ambizioso, pragmatico e tranquillo |

**Biografia** Nasce a Firenze nel 30 ottobre del 1970. Crede fortemente nel digitale e per tale motivo vuole utilizzare un app che sia veloce nell'aggiornare i prezzi dei suoi immobili. Ama lavorare in movimento e vuole che l'app gli permetta di monitorare facilmente gli annunci dei suoi clienti ovunque si trovi.

**Interessi** Marco ama bilanciare al massimo lavoro e piacere, puntando a ottimizzare i processi aziendali per avere più tempo da dedicare alla propria vita sociale. Appassionato di commedia, ama citare film e programmi TV nelle conversazioni quotidiane, rendendo l'ambiente lavorativo più leggero e divertente. La sua energia si manifesta anche nelle attività sociali: è sempre entusiasta delle feste in ufficio e vede il networking come un'occasione per consolidare rapporti e creare nuove opportunità. Al centro dei suoi interessi ci sono le relazioni personali, e proprio per questo desidera delegare il più possibile, così da avere più tempo per amici e famiglia. La gestione e l'innovazione aziendale sono una sua grande passione, motivo per cui investe in strumenti digitali che rendano più efficienti le operazioni quotidiane della sua agenzia, permettendogli di automatizzare compiti ripetitivi e concentrarsi su ciò che conta davvero: godersi la vita.

**Obiettivi** Marco ha un obiettivo ben chiaro in mente: semplificare la gestione della sua agenzia senza rinunciare al controllo sulle operazioni. Per questo, è interessato a creare account per i suoi collaboratori, così da delegare attività operative e concentrarsi su aspetti strategici. Vuole poter monitorare le performance di vendita e i progressi dell'agenzia in tempo reale, direttamente dall'app, per essere sempre aggiornato anche mentre è in movimento. L'accesso immediato a report e statistiche dettagliate è fondamentale per analizzare il mercato e ottimizzare le strategie di vendita, senza dover passare ore davanti al computer. Per lui, la possibilità di controllare le vendite dei suoi immobili e visualizzare informazioni aggiornate in tempo reale tramite un'unica piattaforma digitale è essenziale, perché gli permette di gestire l'agenzia in modo efficiente e, al tempo stesso, godersi la vita senza vincoli.

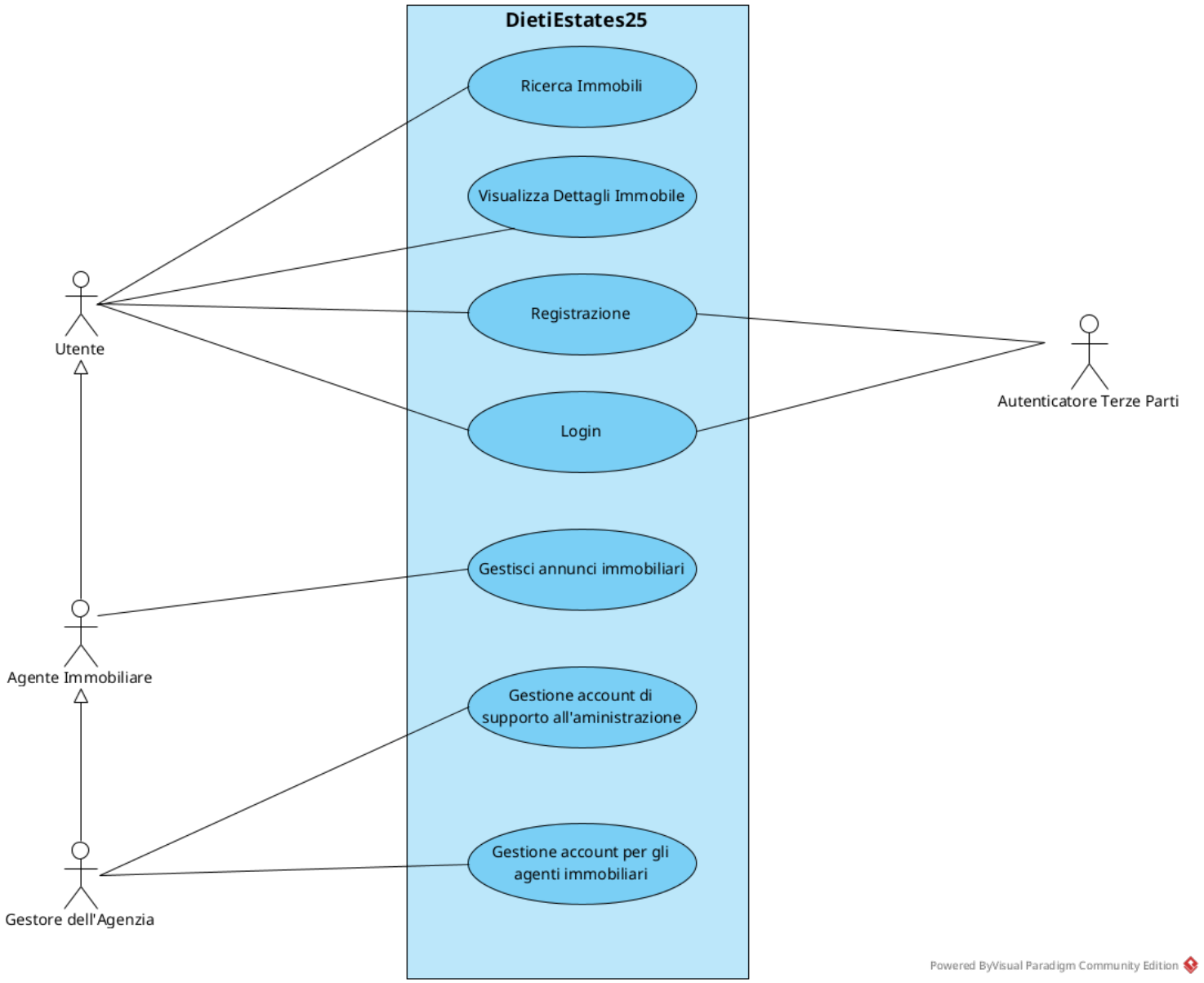
**Frustrazioni/difficoltà** Marco è frustrato dalla quantità di burocrazia che deve gestire ogni giorno e teme che l'agenzia perda clienti a causa dei portali online, che offrono servizi sempre più veloci per cercare immobili. Ogni giorno si trova a fare i conti con contratti, documenti e normative che rallentano tutto. Inoltre, i clienti sono spesso indecisi e i proprietari sopravvalutano i loro immobili, mentre la concorrenza online sembra avere la meglio. Marco sente la pressione di dover tenere il passo con queste nuove tecnologie e teme che, se non cambia, l'agenzia possa restare indietro.

## 1.3 Casi d'Uso e Descrizione dei Requisiti

In questa sezione vengono elencati e descritti in dettaglio i requisiti del sistema DietiEstates, individuati durante la fase di prototipazione sulla base delle specifiche funzionali fornite dal committente.

L'obiettivo di questa fase è stato quello di tradurre le esigenze espresse in termini operativi concreti, definendo le funzionalità che il sistema dovrà offrire per soddisfare le aspettative degli utenti finali e garantire il raggiungimento degli obiettivi progettuali.

I requisiti identificati sono stati analizzati e formalizzati tenendo conto dell'ambiente operativo, dei vincoli tecnici e delle priorità indicate. Essi costituiscono la base per lo sviluppo del sistema e saranno utilizzati come riferimento per le attività di verifica, validazione e collaudo.




### 1.3.1 Descrizione dei requisiti

1. **Performance (Prestazioni):**
   (a) Il sistema deve essere performante, con tempi di risposta rapidi per garantire un'ottima esperienza utente.
   (b) Il motore di ricerca deve essere ottimizzato per gestire grandi volumi di annunci immobiliari, con tempi di elaborazione costanti.
2. **Autenticazione, autorizzazione e gestione degli utenti**
   (a) Il sistema permette a più agenzie di pubblicare inserzioni immobiliari. Ogni installazione di DietiEstates25 include un'utenza di amministrazione per il gestore dell'agenzia immobiliare, che viene creata con credenziali predefinite. Dopo aver effettuato l'accesso, è possibile modificare la password di amministrazione.
   (b) Il sistema deve implementare un meccanismo di autenticazione sicura basato su token JWT (JSON Web Token), da includere nelle richieste alle API protette.
   (c) Ogni utente è identificato univocamente da Nome, Cognome ed Email. L'indirizzo email è considerato **univoco** e utilizzato per l'autenticazione.
   (d) Non è consentita la registrazione di più account con lo stesso indirizzo email. In caso di inserimento di un'email già registrata, il sistema deve notificare l'errore e impedire la creazione.
   (e) Gli utenti sono associati a ruoli che definiscono le autorizzazioni sulle funzionalità del sistema. I ruoli previsti sono:
      - `ADMIN`: amministratore di una agenzia immobiliare (gestione annunci e collaboratori);
      - `AGENT`: collaboratore dell'agenzia immobiliare (gestione annunci, gestione appuntamenti);

- `USER` : utente generico che visualizza annunci e prenotare degli appuntamenti.
- `ADMIN_ASSISTANT`: agente immobiliare con il privilegio di aggiunta, rimozione e promozione.

(f) Gli agenti con ruolo `ADMIN` oppure `ADMIN_ASSISTANT` hanno la facoltà di creare account per agenti immobiliari e di promuovere un (`AGENT`) al ruolo `ADMIN_ASSISTANT`:

- Un agente con tale ruolo **non può** eliminare agenti con il ruolo `ADMIN` o auto-promuoversi a tale ruolo.
- Laddove l'email del nuovo agente immobiliare fosse già presente nel database, il sistema procede alla modifica del ruolo dell'account in questione.
- La promozione ad `ADMIN_ASSISTANT` deve essere confermata con una doppia verifica da parte dell'amministratore.
- Un agente immobiliare può lavorare *per una ed una sola agenzia immobiliare*, non deve essere quindi possibile aggiungere un collaboratore già associato ad altre agenzie immobiliari.

3. **Persistenza dati**

(a) Il sistema deve utilizzare un database relazionale per la gestione delle informazioni persistenti, incluse le entità utenti, annunci, agenzie e ruoli.

(b) Tutte le operazioni critiche devono essere gestite in modo atomico, garantendo consistenza anche in caso di errore o interruzione.

(c) Il sistema deve prevedere il supporto per il salvataggio di formati multimediali standard JPEG, PNG.

4. **Manutenibilità**

(a) Il sistema deve disporre di test automatici.

(b) Gli aggiornamenti del sistema devono essere gestiti tramite un sistema di versioning (es. Git).

5. **User Experience (UX)**

(a) L'interfaccia utente deve essere intuitiva, con un design pulito e coerente per facilitare la navigazione.

(b) Il sistema deve garantire tempi di caricamento rapidi per migliorare l'esperienza utente.

(c) Le azioni comuni (registrazione, login, ricerca immobili) devono essere eseguibili con un numero minimo di interazioni.

(d) Il sistema deve fornire feedback chiaro e immediato per ogni azione dell'utente, inclusi errori e conferme.

6. **Classificazione e visualizzazione annunci:**

(a) Gli annunci devono essere categorizzati in base alla tipologia di transazione (es. "Vendita", "Affitto").

7. **Sicurezza**

(a) Le password degli utenti devono essere salvate usando algoritmi di hashing sicuri (es. bcrypt o Argon2).

## 1.4 FULLY DRESSED USE CASE DIAGRAMS

### 1.4.1 Ricerca di un immobile

| | | | |
|---|---|---|---|
| **Use Case #1** | Ricerca di un immobile | | |
| **Goal in Context** | L'utente desidera effettuare una ricerca per trovare un immobile adatto alle proprie esigenze | | |
| **Preconditions** | Nessuna precondizione, anche un utente non loggato può effettuare ricerche nel sistema | | |
| **Success End Condition** | L'utente individua un elenco di immobili che soddisfano i criteri da lui individuati. | | |
| **Failed End Condition** | Il sistema non restituisce alcun risultato. | | |
| **Primary Actor** | Utente | | |
| **Trigger** | L'utente apre la pagina "Search" presente sulla navbar della pagina 1.1. | | |
| **Main Scenario** | **Step n°** | **Utente** | **Sistema** |

(Continua nella pagina successiva)

Tabella 1.2:

| | | | |
|---|---|---|---|
| | 1 | L'utente apre la pagina "Search" presente sulla navbar della pagina 1.1. | |
| | 2 | | Carica la pagina 1.2 |
| | 3 | L'utente seleziona un punto dalla mappa o cerca un indirizzo dalla barra di ricerca. | |
| | 4 | | Il sistema mostra un marker sulla mappa interattiva per mostrare il punto a partire dal quale avverrà la ricerca |
| | 5 | L'utente seleziona tipologia di annuncio dal dropdown menu che presenta le opzioni AFFITTO e VENDITA. | |
| | 6 | Preme "Cerca". | |
| | 7 | | La pagina viene caricata con tutti gli immobili che corrispondono ai parametri di ricerca. |
| | 8 | Seleziona un immobile per poterne visualizzare la pagina contenente i dettagli completi. | |
| | 9 | | Mostra la pagina dell'immobile. Fine del caso d'uso. |
| **Extension # 1: Ricerca con parametri avanzati** | **Step n°** | **Utente** | **Sistema** |
| | 1.6 | L'utente clicca su "Mostra filtri avanzati". | |
| | 1.7 | | Viene mostrato un form di ricerca con filtri avanzati come in 1.3. |
| | 1.8 | L'utente seleziona i campi opzionali desiderati. | |
| | 1.9 | | Lo scenario riprende dallo step 7 del Main Scenario |
| **Extension # 2: La ricerca non produce risultati** | **Step n°** | **Utente** | **Sistema** |
| | 2.5 | | Mostra un avviso "La ricerca non ha prodotto risultati". L'utente può decidere di ripetere la ricerca oppure concludere il caso d'uso. |
| **Extension # 3: L'utente non seleziona un indirizzo o un punto dalla mappa** | **Step n°** | **Utente** | **Sistema** |
| | 3.5 | L'utente preme su "Cerca" senza aver specificato una posizione attorno alla quale cercare gli annunci. | |
| | 3.6 | | Mostra un avviso "Seleziona una posizione sulla mappa o cerca un indirizzo per procedere.". L'utente può decidere di ripetere la ricerca oppure concludere il caso d'uso. |

Tabella 1.2:

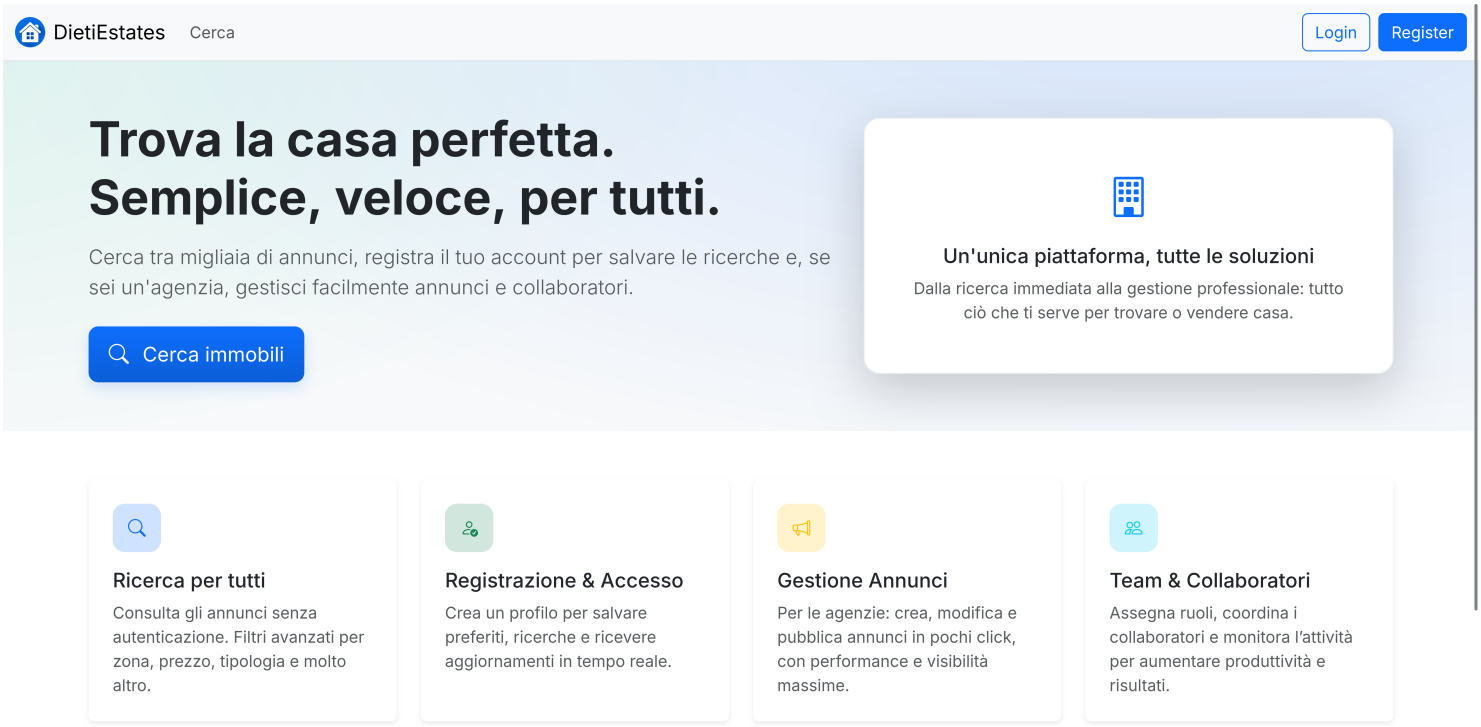



Figura 1.1: Homepage per un utente non autenticato

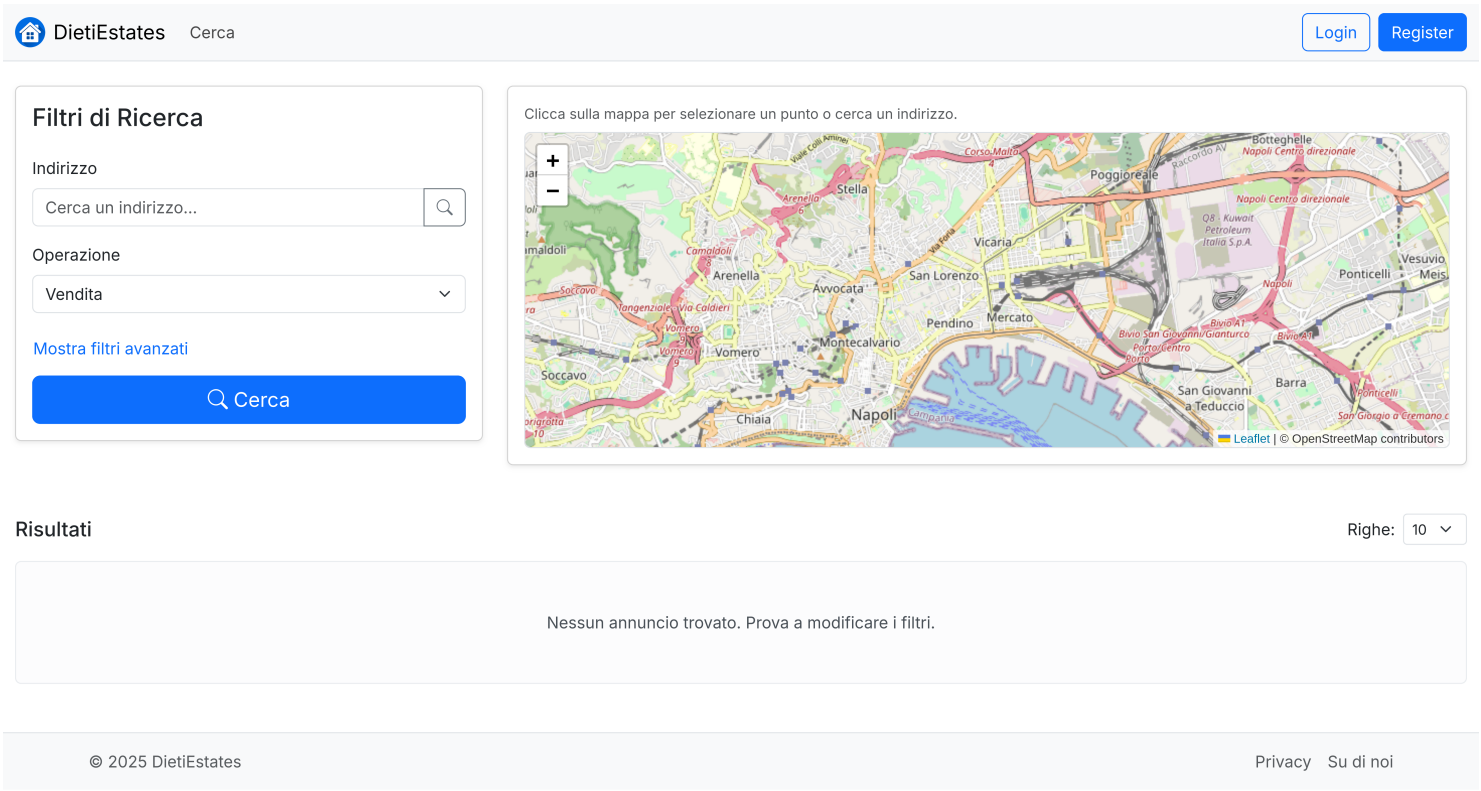



Figura 1.2: Pagina per la ricerca di base

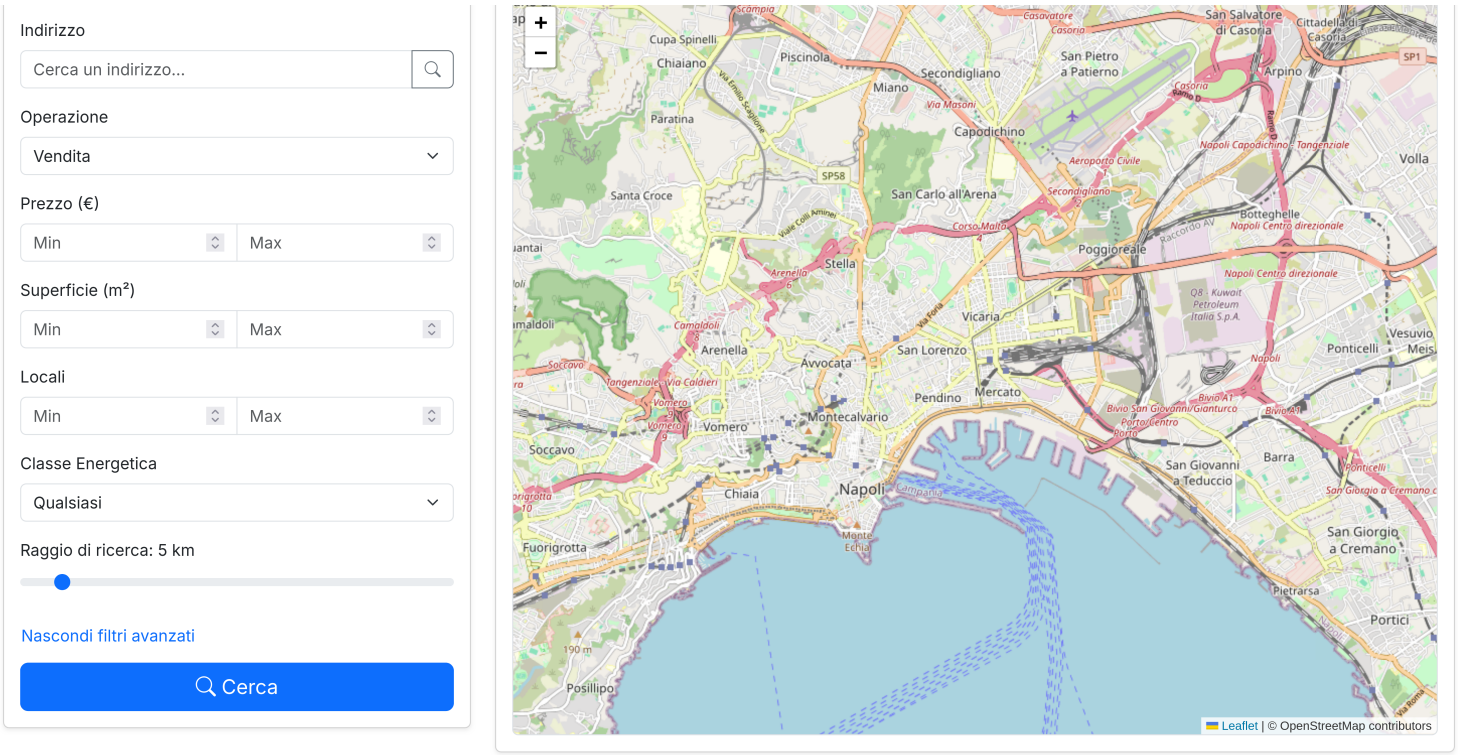



Figura 1.3: Pagina per la ricerca di base

### 1.4.2 Caricamento di un annuncio immobiliare

| | | | |
|---|---|---|---|
| **Use Case #2** | Aggiunta annuncio immobiliare | | |
| **Goal in context** | Aggiungere un nuovo annuncio | | |
| **Precontions** | Un annuncio può essere creato da un agente immobiliare o un gestore di un'agenzia immobiliare precedentemente autenticato. | | |
| **Success End Condition** | L'annuncio viene aggiunto al database e visibile nella pagina degli annunci listati dall'agente, oltre che raggiungibile mediante la pagina di ricerca | | |
| **Failed End Condition** | L'annuncio non viene inserito | | |
| **Primary Actor** | Agente immobiliare o gestore di un'agenzia immobiliare | | |
| **Trigger** | Viene cliccata la CTA "Aggiungi annuncio" dalla schermata 1.4 | | |
| **Main Scenario** | **Step n°** | **Agente** | **Sistema** |
| | 1 | Clicca sulla CTA "Aggiungi annuncio" nella schermata 1.4 | |
| | 2 | | Apre schermata 1.5 |
| | 3 | Aggiunge titolo dell'annuncio | |
| | 4 | Aggiunge descrizione dell'annuncio | |
| | 5 | Aggiunge tipologia dell'annuncio, se vendita oppure affitto | |
| | 6 | Aggiunge prezzo dell'annuncio | |
| | 7 | Aggiunge immagini dell'immobile | |

(Continua nella pagina successiva)

Tabella 1.3:

| | | | |
|---|---|---|---|
| | 8 | Inserisce indirizzo e preme “Cerca” | |
| | 9 | | Chiama servizio esterno Nominatim per prelevare coordinate dell’indirizzo |
| | 10 | | Mostra marker sulla mappa interattiva per indicare la posizione dell’immobile |
| | 11 | Preme “Salva” | |
| | 12 | | Valida il form. |
| | 13 | | Salva l’annuncio nel database. |
| | 14 | | Reindirizza l’utente alla pagina 1.4 mostrando una nuova card per l’annuncio appena creato. Fine del caso d’uso. |
| **Extension #1: Utente preme “Salva” senza aver caricato una foto** | **Step n°** | **Agente** | **Sistema** |
| | 1.13 | | Mostra messaggio di errore “Caricare almeno un’immagine” |
| **Extension #2: Utente preme salva senza aver compilato campi obbligatori** | **Step n°** | **Agente** | **Sistema** |
| | 2.12 | | Mostra messaggio di errore “Compila campi obbligatori” |
| **Extension #3: Utente preme “Annulla”** | **Step n°** | **Agente** | **Sistema** |
| | 3.2 | | Mostra messaggio di conferma |
| | 3.3 | Preme “Conferma” | |
| | 3.4 | | Reindirizza l’utente alla pagina 1.4 senza aggiungere l’annuncio. |

Tabella 1.3:

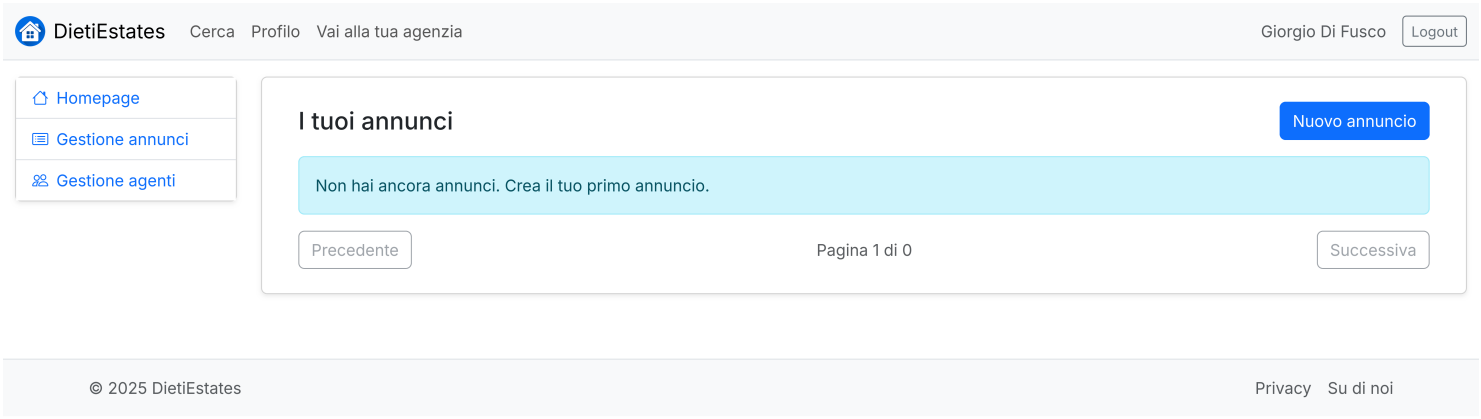



Figura 1.4: Pagina per la gestione degli annunci immobiliari

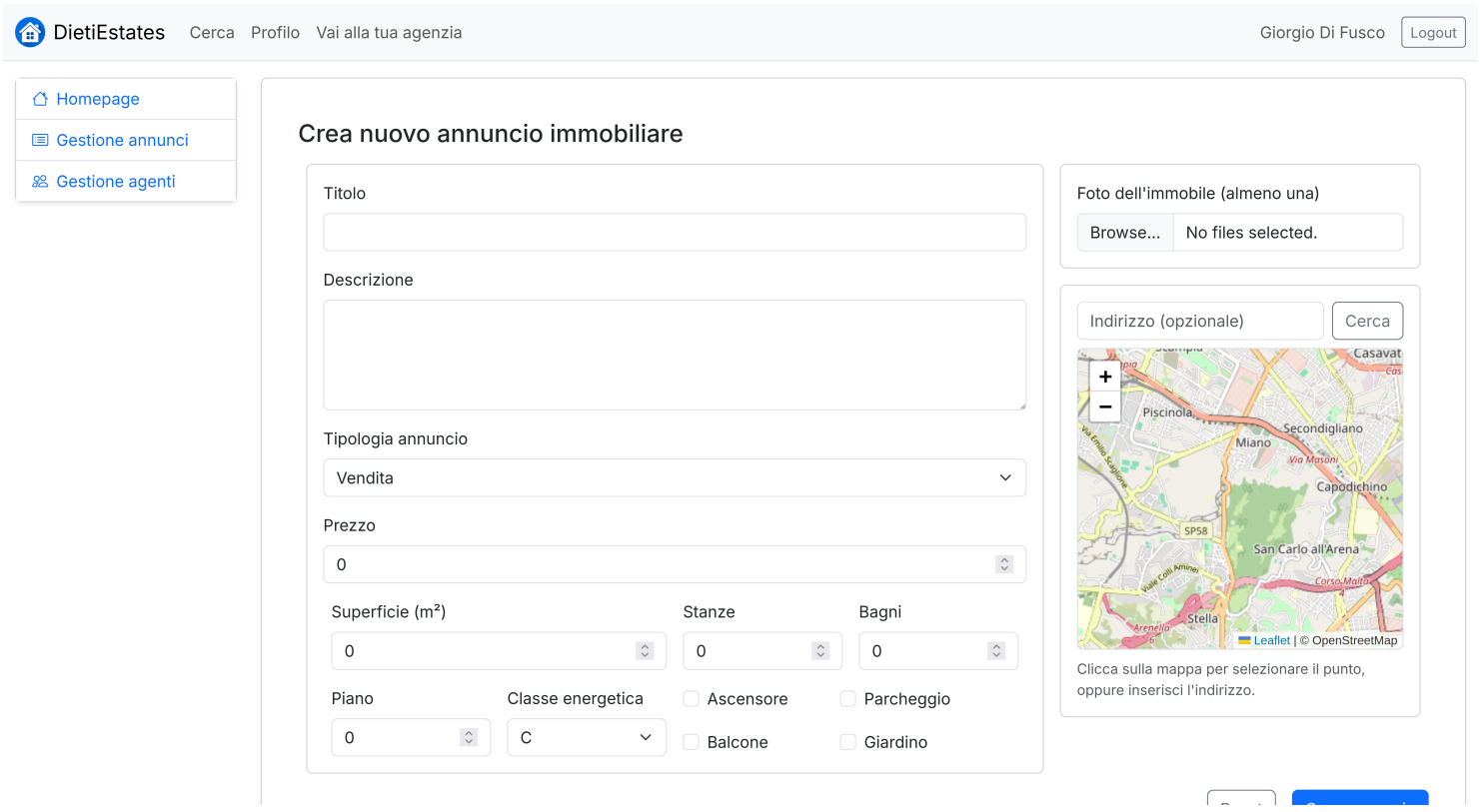



Figura 1.5: Pagina per l'aggiunta per un nuovo annuncio immobiliare

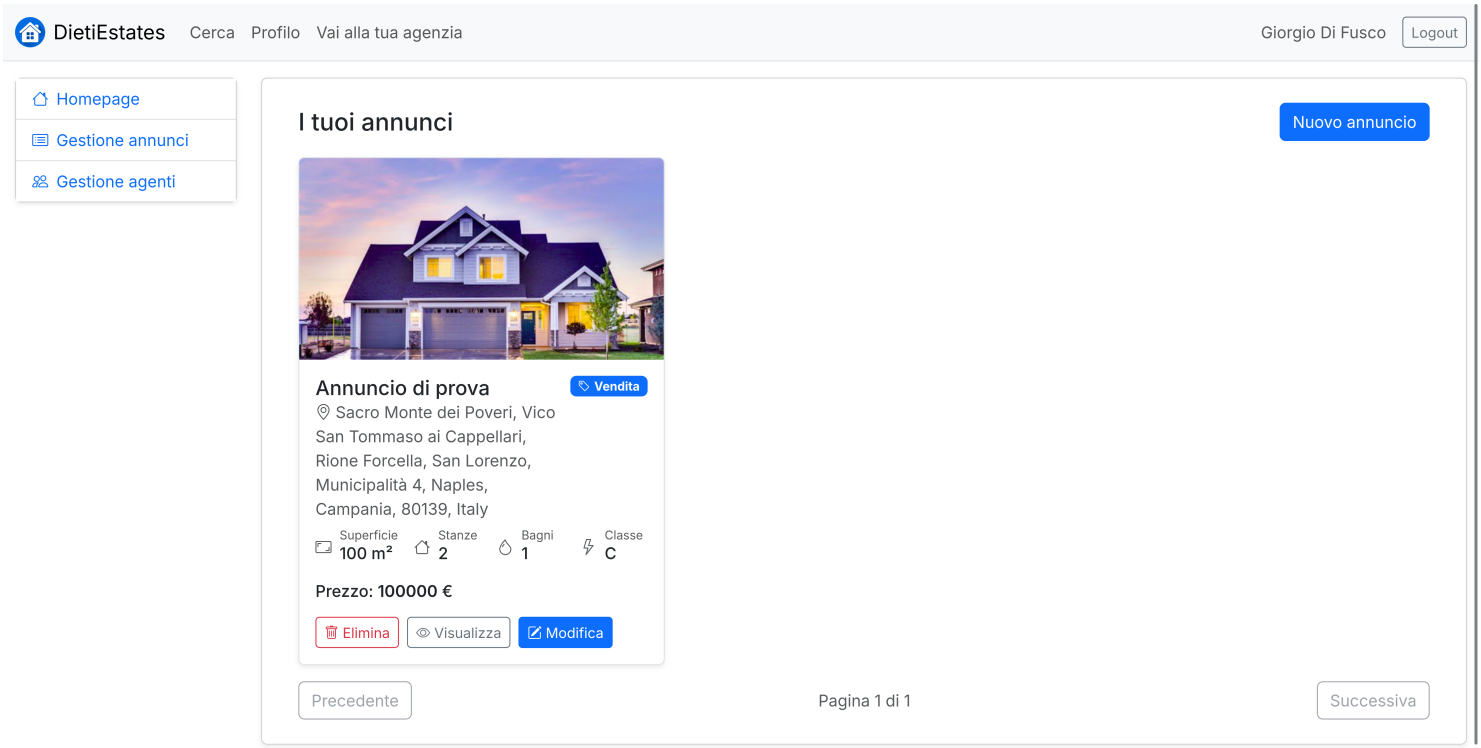



Figura 1.6: Pagina con la card aggiornata

### 1.4.3 Aggiunta di un agente immobiliare

| | | | |
|---|---|---|---|
| **Use Case #3** | Aggiunta di un nuovo agente immobiliare | | |
| **Goal in Context** | Il gestore di un'agenzia immobiliare vuole aggiungere un nuovo agente | | |
| **Preconditions** | L'utente possiede ruolo `ROLE_ADMIN` oppure `ROLE_ADMIN_ASSISTANT` | | |
| **Success End Condition** | Il collaboratore viene associato all'agenzia | | |
| **Failed End Condition** | Non viene inserita l'associazione utente-agenzia. | | |
| **Primary Actor** | Gestore di una agenzia immobiliare | | |
| **Trigger** | L'amministratore, dalla schermata 1.7, clicca su "Aggiungi collaboratore" | | |
| **Main Scenario** | **Step n°** | **Agente** | **Sistema** |
| | 1 | Clicca su "Aggiungi collaboratore" | |
| | 2 | | Carica schermata 1.8 |
| | 3 | Inserisce nome, cognome ed email dell'agente che intende inserire. | |
| | 4 | Preme su "Salva" | |
| | 5 | | Valida i campi |
| | 6 | | Mostra messaggio di successo |
| | 7 | | Reindirizza l'utente alla schermata 1.7 |
| | 8 | | Il sistema aggiunge un account relativo all'agente con gli opportuni permessi nel database e con una password predefinita. Collega l'agente all'agenzia immobiliare |

(Continua nella pagina successiva)

Tabella 1.4:

| | | | |
|---|---|---|---|
| | 9 | | Invia una email all'agente con un codice di attivazione e l'invito a modificare la propria password |
| **Extension #1: Email agente già presente** | **Step n°** | **Agente** | **Sistema** |
| | 1.5 | | Il sistema individua l'utente associato alla email inserita |
| | 1.6 | | Assegna ruolo `ROLE_AGENT` all'utente e lo associa all'agenzia dell'utente che lo ha inserito |
| | 1.7 | | Invia una email all'utente promosso al ruolo di agente per comunicare la promozione |

Tabella 1.4:

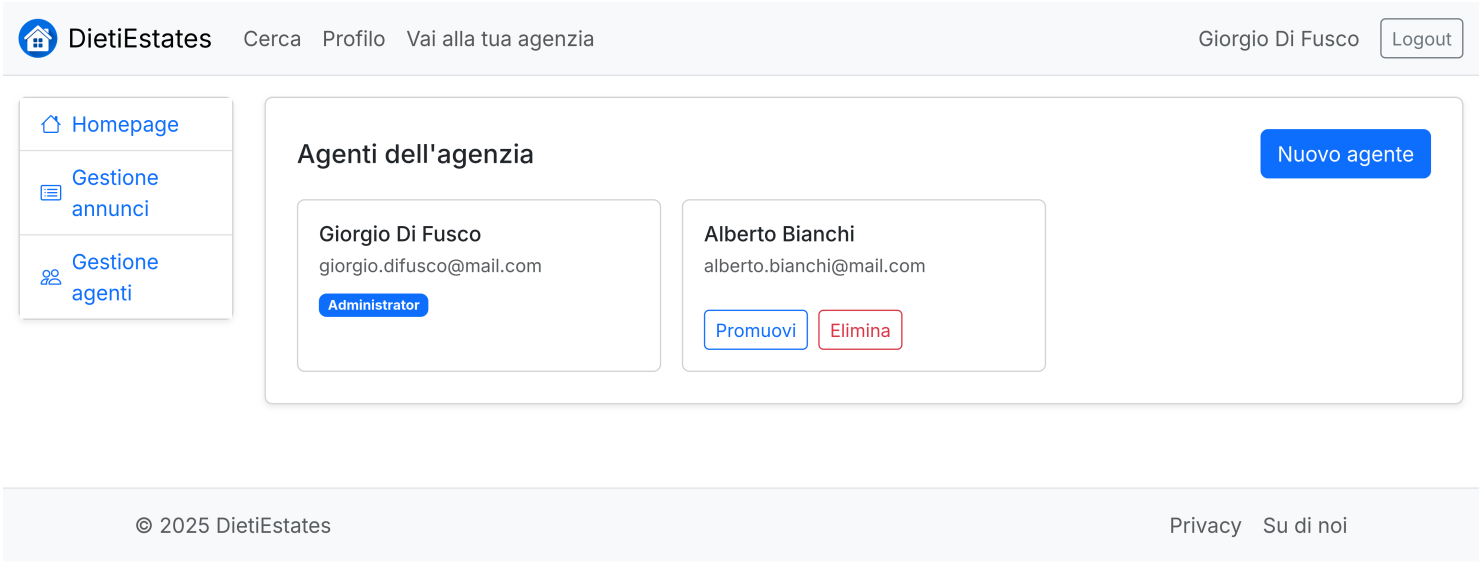



Figura 1.7: Dashboard gestione agenti immobiliari

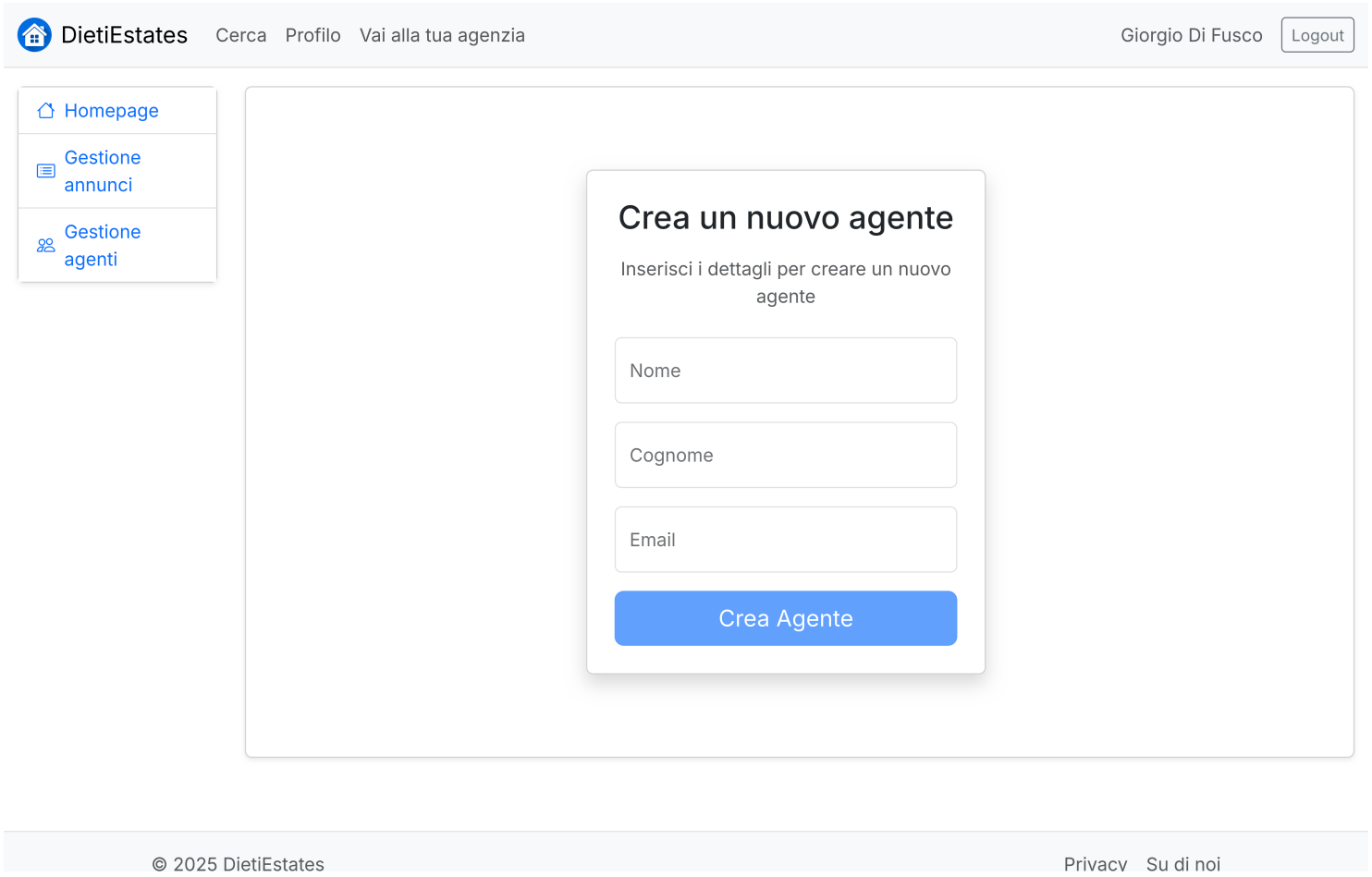



Figura 1.8: Form aggiunta agenti immobiliari

### 1.4.4 Promozione di un agente immobiliare

| | | | |
|---|---|---|---|
| **Use Case #4** | Promozione di un agente | | |
| **Goal in Context** | Il gestore di un'agenzia immobiliare vuole promuovere un agente | | |
| **Preconditions** | L'agente da promuovere appartiene all'agenzia | | |
| **Success End Condition** | Il collaboratore viene registrato nel sistema ed è visibile nell'elenco utenti | | |
| **Failed End Condition** | Non viene inserito alcun nuovo utente | | |
| **Primary Actor** | Gestore di una agenzia immobiliare | | |
| **Trigger** | L'amministratore, dalla dashboard 1.7, clicca su "Promuovi" | | |
| **Main Scenario** | **Step n°** | **Agente** | **Sistema** |
| | 1 | Seleziona la card dell'agente e clicca su "Promuovi" | |
| | 2 | | Mostra schermata di conferma. |
| | 3 | Clicca "Conferma" | |
| | 4 | | Assegna ruolo `ROLE_ADMIN_ASSISTANT` all'agente |
| | 5 | | Invia email all'agente promosso per comunicargli della promozione |
| | 6 | | Aggiorna la card dell'agente promosso. |

Tabella 1.5:

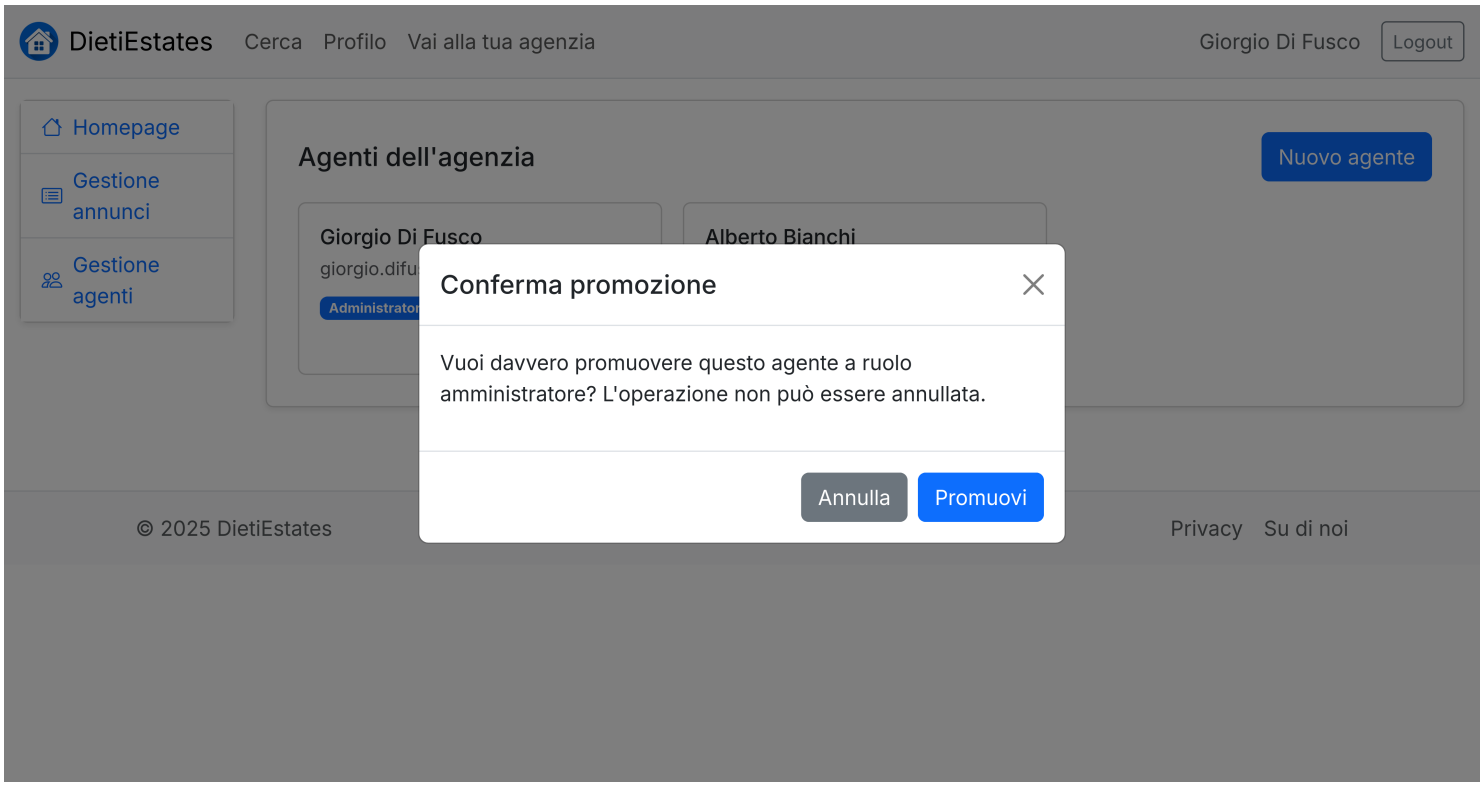



Figura 1.9: Finestra modale per conferma promozione

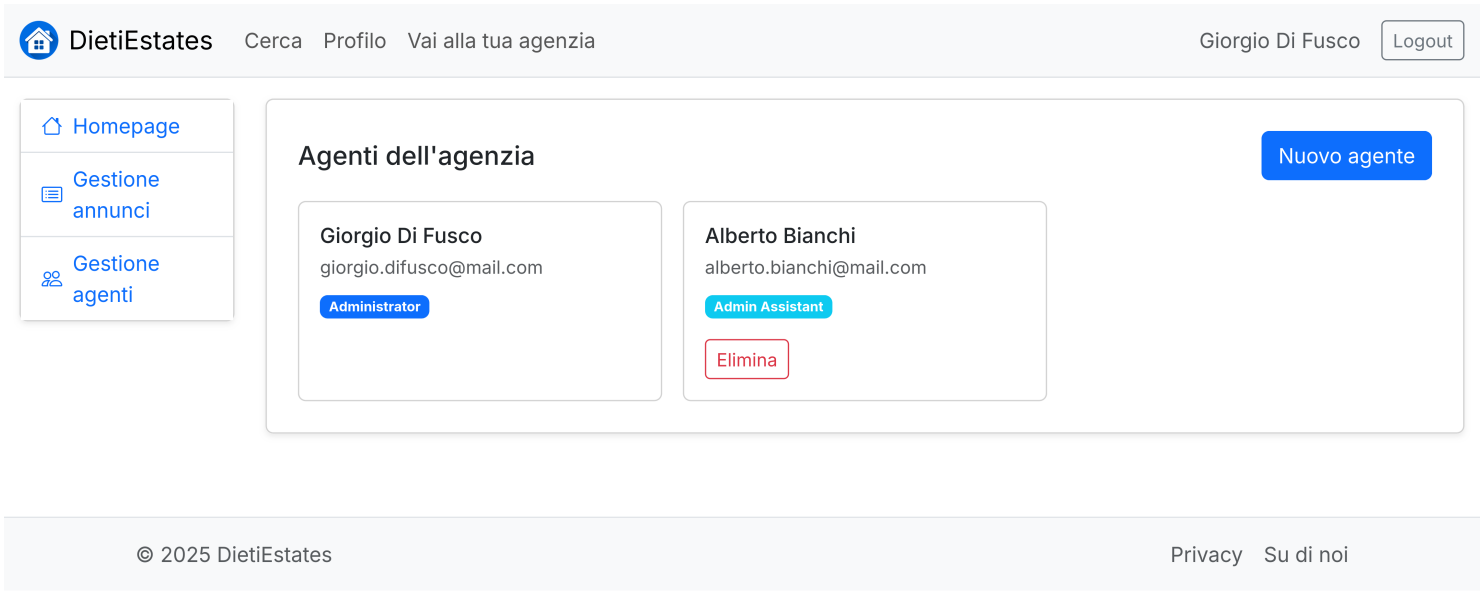



Figura 1.10: Card agente aggiornata col nuovo ruolo

# Architettura e Design del Sistema

In questo capitolo vengono illustrate in dettaglio le scelte architetturali, le tecnologie adottate e i principi di design che guidano lo sviluppo del sistema DietiEstates. L'obiettivo è fornire una visione chiara della struttura software, giustificando le decisioni prese in ottica di manutenibilità, scalabilità e robustezza, secondo le migliori pratiche dell'ingegneria del software.

## 2.1 Architettura del Sistema

Il sistema è stato progettato seguendo un'architettura multi-tier, specificamente un'architettura a tre livelli (Three-Tier Architecture), che separa nettamente la logica di presentazione, la logica di business e la gestione dei dati. Questo approccio favorisce la modularità, il disaccoppiamento tra i componenti e la possibilità di sviluppare, manutenere e scalare i singoli livelli in modo indipendente.

### 2.1.1 Livello di Presentazione (Presentation Tier)

Il livello di presentazione è costituito da un'applicazione web di tipo Single-Page Application (SPA), sviluppata con il framework Angular. Questa applicazione viene eseguita interamente nel browser dell'utente e ha la responsabilità di renderizzare l'interfaccia grafica, gestire le interazioni dell'utente e comunicare con il livello di business tramite chiamate API. La scelta di una SPA permette di offrire un'esperienza utente fluida e reattiva, simile a quella di un'applicazione desktop, minimizzando i tempi di attesa grazie al caricamento asincrono dei dati.

### 2.1.2 Livello di Business (Application Tier)

Il cuore del sistema risiede nel livello di business, un'applicazione backend sviluppata in Java utilizzando il framework Spring Boot. Questo livello espone un'interfaccia RESTful API sicura, attraverso la quale il client (l'applicazione Angular) può accedere alle funzionalità del sistema. Le responsabilità di questo tier includono:

- **Gestione della Logica di Business:** Implementazione di tutte le logiche applicative, come la gestione degli annunci immobiliari, la gestione degli utenti e dei loro ruoli.
- **Autenticazione e Autorizzazione:** messa in sicurezza degli endpoint dell'API tramite Spring Security, gestendo l'autenticazione degli utenti (login) e l'autorizzazione basata sui ruoli (utente, agente, amministratore).
- **Comunicazione con il Livello Dati:** Interazione con il database PostgreSQL per il salvataggio e il recupero persistente delle informazioni.

La comunicazione tra il Presentation Tier e l'Application Tier avviene tramite il protocollo HTTP/S, con scambio di dati in formato JSON (JavaScript Object Notation), uno standard leggero e facilmente interpretabile da entrambi i sistemi.

### 2.1.3 Livello Dati (Data Tier)

Il livello dati è responsabile della persistenza delle informazioni. È costituito da un database relazionale (ad esempio PostgreSQL) che memorizza tutti i dati dell'applicazione, come le anagrafiche degli utenti, i dettagli degli immobili e le agenzie immobiliari con i loro agenti.

L'accesso a questo livello è gestito esclusivamente dall'Application Tier, che si avvale di un Object-Relational Mapper (ORM) come Hibernate, tramite l'astrazione offerta da Spring Data JPA. L'uso di un ORM astrae la complessità del linguaggio SQL,

permettendo agli sviluppatori di interagire con il database attraverso oggetti Java (Entity), migliorando la manutenibilità e la portabilità del codice.

## 2.2 Tecnologie Adottate

La selezione delle tecnologie è stata guidata da criteri di maturità, supporto della comunità, produttività e aderenza agli standard di mercato.

### 2.2.1 Backend

- **Java:** Linguaggio di programmazione robusto, orientato agli oggetti e indipendente dalla piattaforma, ampiamente utilizzato per lo sviluppo di applicazioni enterprise complesse e performanti.
- **Spring Boot:** Framework che semplifica drasticamente lo sviluppo di applicazioni basate su Spring. È stato scelto per la sua capacità di auto-configurazione, la gestione integrata delle dipendenze e l'ecosistema completo che include moduli per la sicurezza (Spring Security), l'accesso ai dati (Spring Data JPA) e la creazione di API REST (Spring Web).
- **JPA/Hibernate:** Standard per la persistenza dei dati in Java. Hibernate, come implementazione JPA, gestisce la mappatura degli oggetti Java alle tabelle del database, semplificando le operazioni CRUD e le query complesse.
- **Maven:** Strumento di build automation e gestione delle dipendenze, utilizzato per definire la struttura del progetto, compilare il codice sorgente e gestire le librerie di terze parti.
- **Docker:** Piattaforma di containerizzazione utilizzata tramite Docker Compose per definire e orchestrare l'ambiente di sviluppo e produzione, garantendo coerenza e replicabilità tra le diverse macchine e semplificando il deployment.

### 2.2.2 Approfondimento: Sicurezza con Spring Security e OpenID Connect

Oltre alle scelte architetturali di alto livello, sono stati adottati specifici pattern di progettazione e configurazioni di sicurezza avanzate per garantire la robustezza, la manutenibilità e la sicurezza del sistema.

Per garantire un elevato livello di sicurezza nell'accesso alle risorse del sistema, è stato adottato Spring Security in combinazione con il protocollo OpenID Connect (OIDC). Questa scelta consente di implementare un sistema di autenticazione e autorizzazione robusto e scalabile, sfruttando le funzionalità offerte da provider di identità esterni.

Spring Security fornisce un framework completo per la gestione della sicurezza delle applicazioni Java, offrendo funzionalità come l'autenticazione degli utenti, l'autorizzazione basata sui ruoli e la protezione contro le vulnerabilità comuni (es. CSRF, XSS). Integrando OIDC, il sistema può delegare l'autenticazione a un Identity Provider (IdP) esterno, che gestisce in modo sicuro le credenziali degli utenti e fornisce token di accesso standardizzati. Questo approccio presenta diversi vantaggi:

- **Sicurezza Migliorata:** La gestione delle credenziali è affidata a un provider specializzato, riducendo il rischio di vulnerabilità legate alla gestione interna delle password.
- **Scalabilità:** L'utilizzo di un IdP esterno consente di scalare facilmente il sistema di autenticazione senza dover implementare soluzioni personalizzate.
- **Esperienza Utente Unificata:** Gli utenti possono utilizzare le stesse credenziali per accedere a più applicazioni, migliorando l'usabilità e riducendo la necessità di gestire più account.

L'implementazione di Spring Security con OIDC prevede la configurazione del client OIDC all'interno dell'applicazione Spring Boot, la definizione delle regole di accesso alle risorse protette e la gestione dei token di accesso per autorizzare le richieste API.

**Il design pattern Factory per la gestione dei vari providers** Per gestire l'autenticazione tramite provider esterni (es. Google) secondo i protocolli OAuth2 e OpenID Connect, è stato implementato il **Factory Pattern**. Questa scelta di design è cruciale per disaccoppiare la logica di gestione dell'autenticazione dalle implementazioni specifiche di ciascun provider.

Il cuore di questo pattern è rappresentato dall'interfaccia `OAuth2UserInfo`, che definisce un contratto comune per l'estrazione degli attributi di un utente (come ID, nome, email) a prescindere dal provider. Per ogni provider supportato, viene creata una classe concreta (es. `GoogleOAuth2UserInfo`) che implementa questa interfaccia, sapendo come mappare la struttura dati, spesso eterogenea, restituita da quel provider.

La classe `OAuth2UserInfoFactory` agisce come "fabbrica": il suo metodo principale riceve in input il nome del provider e la mappa di attributi dell'utente, e restituisce l'istanza della classe concreta appropriata. In questo modo, i servizi che gestiscono il processo di login (`CustomOAuth2UserService` e `CustomOidcUserService`) non devono contenere logica condizionale (if/else o switch) per i diversi provider. Essi interagiscono esclusivamente con l'astrazione `OAuth2UserInfo`, favorendo il riutilizzo del codice e rendendo il sistema facilmente estensibile: per supportare un nuovo provider, è sufficiente creare una nuova implementazione

dell'interfaccia e aggiornare la factory, senza modificare i servizi esistenti.

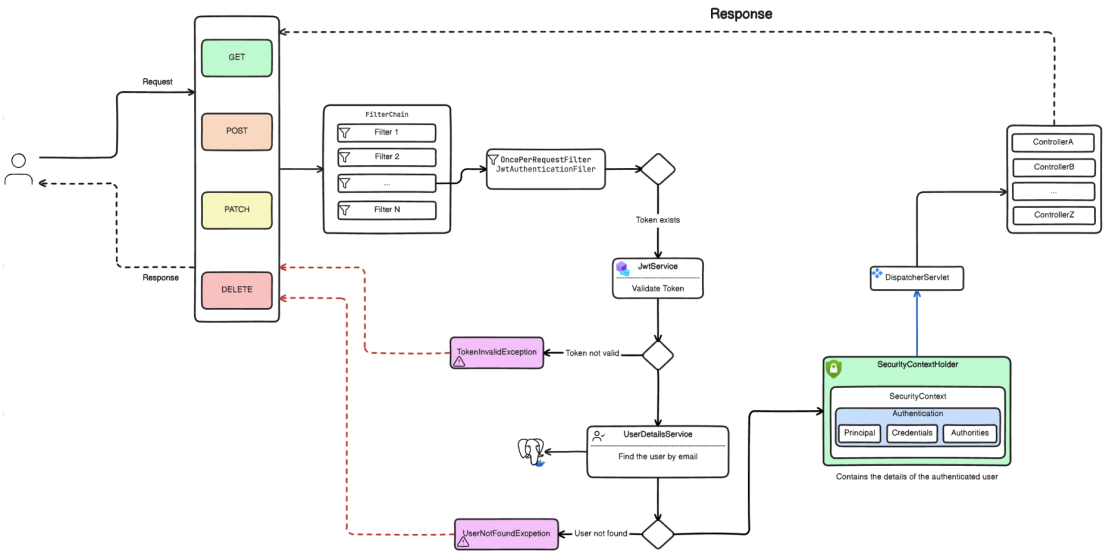
**Gestione della sicurezza con la Security Filter Chain** La sicurezza dell'applicazione backend è gestita da Spring Security attraverso una catena di filtri (**Security Filter Chain**). Questo approccio, che implementa il pattern Chain of Responsibility, permette di applicare una serie di controlli di sicurezza a ogni richiesta HTTP in arrivo.

La configurazione, definita nella classe `SecurityFilterConfig`, stabilisce l'ordine e il comportamento di questi filtri. Le principali responsabilità configurate includono:

- **Autorizzazione degli Endpoint:** Viene definita una politica di accesso granulare per i diversi percorsi dell'API. Ad esempio, gli endpoint relativi all'autenticazione (es. `/api/auth/**`) sono resi pubblici, mentre altre aree richiedono ruoli specifici (es. `/api/admin/**` richiede il ruolo di `ADMIN`).
- **Gestione delle Sessioni:** L'applicazione è configurata per essere *stateless*, ovvero non viene creata alcuna sessione utente sul server. Questa è una scelta fondamentale per le API RESTful, in quanto garantisce la scalabilità e semplifica il bilanciamento del carico. L'autenticazione di ogni richiesta è affidata a un token.
- **Integrazione OIDC e OAuth2:** La catena è configurata per intercettare e gestire i flussi di login con provider esterni, delegando il processo ai servizi specifici come `CustomOidcUserService`.
- **Filtri Personalizzati:** È stato inserito un filtro custom, `AuthFilter`, posizionato strategicamente all'interno della catena. Questo filtro ha il compito di ispezionare l'header `Authorization` di ogni richiesta, estrarre e validare il JSON Web Token (JWT) e, in caso di successo, popolare il `SecurityContext` di Spring. Questo passaggio è ciò che permette al resto dell'applicazione di identificare l'utente autenticato e i suoi ruoli per le successive richieste API.

**Access token JWT e Refresh Tokens** Per garantire un'autenticazione sicura e scalabile, il sistema utilizza JSON Web Tokens (JWT) come access token per autorizzare le richieste API. I JWT sono token compatti e auto-contenuti che trasportano informazioni sull'utente e sui suoi privilegi in modo sicuro. Quando un utente effettua il login tramite OIDC, riceve un access token JWT che deve essere incluso nelle intestazioni delle richieste API per accedere alle risorse protette. Questo approccio consente di evitare la necessità di mantenere sessioni lato server, migliorando la scalabilità dell'applicazione.

Accanto agli access token, il sistema implementa anche l'uso di refresh token. I refresh token sono token a lunga durata che consentono agli utenti di ottenere nuovi access token senza dover effettuare nuovamente il login. Quando un access token scade, l'applicazione può utilizzare il refresh token per richiedere un nuovo access token all'IdP, migliorando l'esperienza utente e mantenendo la sicurezza delle sessioni.




### 2.2.3 Frontend

- **Angular:** Framework client-side basato su TypeScript, sviluppato da Google. È stato scelto per la sua architettura a componenti, che promuove la riusabilità del codice, e per il suo potente sistema di dependency injection, che facilita la scrittura di codice modulare e testabile.
- **TypeScript:** Superset di JavaScript che introduce la tipizzazione statica. Il suo utilizzo migliora la robustezza del codice, previene errori comuni in fase di sviluppo e aumenta la manutenibilità a lungo termine del progetto.
- **OpenAPI Generator:** Strumento fondamentale per l'integrazione tra frontend e backend. A partire dalla specifica OpenAPI (precedentemente nota come Swagger) esposta dal backend, questo strumento genera automaticamente un client API TypeScript completo e type-safe. Questo approccio riduce drasticamente il rischio di disallineamenti e è un pilastro per lo sviluppo contract-first.

- **HTML5 e CSS3:** Standard web utilizzati per la strutturazione semantica delle pagine e per la loro stilizzazione, garantendo compatibilità con i browser moderni e la possibilità di implementare design responsive.

## 2.3 Design dell'Interfaccia Utente (UI)

Il design dell'interfaccia utente è stato concepito per essere pulito, intuitivo e funzionale, con l'obiettivo di massimizzare l'usabilità per le diverse categorie di utenti (visitatori, clienti, agenti e amministratori).

- **Approccio User-Centric:** Ogni elemento dell'interfaccia è stato pensato mettendo al centro le esigenze dell'utente. La navigazione è stata strutturata in modo logico e coerente, permettendo un facile accesso alle funzionalità principali come la ricerca di immobili, la gestione del proprio profilo e, per gli utenti privilegiati, la gestione degli annunci e degli agenti.
- **Coerenza Visiva e Riusabilità:** L'adozione di un'architettura a componenti, nativa in Angular, ha permesso di creare elementi riutilizzabili (es. le card degli annunci, i campi dei form, la navbar). Questo non solo accelera lo sviluppo, ma garantisce anche una forte coerenza visiva e comportamentale in tutta l'applicazione, rendendo l'esperienza utente più prevedibile e confortevole.
- **Responsive Design:** L'interfaccia è stata progettata con un approccio "mobile-first", garantendo che l'applicazione sia perfettamente fruibile su un'ampia gamma di dispositivi, dai desktop agli smartphone. L'utilizzo di tecniche CSS moderne come Flexbox e Grid consente al layout di adattarsi fluidamente alle diverse dimensioni dello schermo.
- **Feedback all'Utente:** Il sistema fornisce un feedback visivo costante per ogni azione intrapresa dall'utente. Indicatori di caricamento (spinner) durante le chiamate asincrone, messaggi di successo o di errore chiari e validazione contestuale dei form sono tutti elementi implementati per migliorare la comunicazione tra il sistema e l'utente, riducendo l'incertezza e migliorando la percezione di reattività dell'applicazione.

L'ispirazione per il layout e l'esperienza utente è tratta dai principali portali immobiliari moderni, cercando di adottare convenzioni già familiari agli utenti per ridurre la curva di apprendimento.

## 2.4 Descrizione dello schema per la persistenza dei dati

### 2.4.1 Panoramica e vincoli generali

Lo schema utilizza PostgreSQL come motore relazionale e sfrutta estensioni quali `postgis` per la gestione dei dati geografici. Le entità principali sono modellate tramite tabelle normalizzate con vincoli di integrità (PK, FK e UNIQUE) e check constraint per valori enumerati (es. classe energetica, tipo di operazione).

Sequenze dedicate gestiscono gli identificatori delle tabelle principali (users, role, listing, listing_image, ecc.) per garantire integrità e comportamento prevedibile nelle operazioni di insert.

### 2.4.2 Tabelle principali

- **agencies**: rappresenta le agenzie immobiliari (id, nome, email, telefono).
- **users**: anagrafica utenti con campi di stato (enabled, account_locked), riferimento opzionale a `agency_id` e controlli sul provider di autenticazione. Le associazioni many-to-many verso **role** sono gestite tramite la tabella di join **users_roles**.
- **role**: definisce i ruoli applicativi (es. ROLE_AGENT, ROLE_ADMIN, ROLE_ADMIN_ASSISTANT) con vincolo di unicità sul nome.
- **listing**: entità principale per gli annunci immobilari. Oltre agli attributi descrittivi (title, description, address, price, rooms, bathrooms, area, energy_class, listing_operation) include un campo spaziale `coordinate` di tipo `geometry(Point,4326)` per memorizzare la posizione geografica del bene.
- **listing_image**: immagini associate ad un annuncio (url, sort_order, created_at, listing_id FK), utilizzata per gestire gallerie e miniature.
- **refresh_tokens** e **otp_token**: tabelle per la gestione di token di autenticazione e OTP.

Gli indici e i vincoli sono definiti per garantire performance e coerenza (es. UNIQUE su email degli utenti, UNIQUE su nome agenzia).

### 2.4.3 Funzioni e logica lato database

Nel file di schema sono presenti stored procedures/functions utili a incapsulare logiche di business che devono essere eseguite atomically lato DB:

- `revoke_agent(user_id INTEGER)`: funzione PL/pgSQL che rimuove ruoli associati a un utente (es. `ROLE_AGENT`, `ROLE_ADMIN_ASSISTANT`) e azzera il campo `agency_id` dell'utente. Questo tipo di funzione è utile per eseguire operazioni di revoca in modo trasparente e coerente, mantenendo la storia delle modifiche tramite l'aggiornamento del campo `last_modified_date`.
- `get_listings_by_agent(p_agent_id INT)`: funzione che restituisce gli annunci associati a un agente, estraendo anche coordinate (lon/lat) e la URL della prima immagine come `thumbnail_url`. Fornisce un'interfaccia semplice e performante per le API che devono elencare gli annunci di un agente.

Queste funzioni permettono di centralizzare controlli e operazioni ripetute, riducendo la complessità delle transazioni lato applicazione e favorendo coerenza dei dati.

### 2.4.4 Motivazione per l'adozione di PostGIS

PostGIS estende PostgreSQL con tipi spaziali (`geometry`, `geography`) e una ricca collezione di funzioni spaziali per operazioni geometriche e geografiche (intersezioni, distanze, buffer, ecc.). Le motivazioni principali della scelta sono:

- **Ricchezza funzionale**: PostGIS fornisce operatori come `ST_DWithin`, `ST_Distance`, `ST_Transform` e funzioni per estrarre coordinate (`ST_X`, `ST_Y`), fondamentali per query di ricerca per prossimità e per visualizzazione su mappe.
- **Performance e indicizzazione**: gli indici spaziali GIST consentono ricerche molto efficienti su grandi insiemi di punti; l'uso di un indice GIST su `listing(coordinate)` permette di limitare rapidamente il numero di geometrie candidate prima di calcolare distanze precise.
- **Precisione geografica**: PostGIS supporta SRID standard (es. 4326) e tipi `geography` per calcoli sulla superficie terrestre che considerano la curvatura della Terra, utile quando le ricerche devono essere espresse in metri.

Per questi motivi PostGIS offre il miglior compromesso tra semplicità d'uso, portata funzionale e performance per le operazioni geospaziali richieste dal prodotto.

### 2.4.5 Uso di PostGIS nelle query di ricerca per raggio

Le ricerche "vicino a una posizione" sono implementate con le funzioni spaziali come mostrato nel seguente listato:

```
-- Punto di ricerca: lon/lat in WGS84
SELECT l.*, ST_X(l.coordinate) AS longitude, ST_Y(l.coordinate) AS latitude
FROM listing l
WHERE ST_DWithin(
    l.coordinate::geography,
    ST_SetSRID(ST_MakePoint(:lon, :lat), 4326)::geography,
    :radius_meters
);
```

Descrizione:

- La conversione a `geography` consente di esprimere il raggio in metri con precisione geodetica.
- `ST_DWithin` è efficiente quando è supportato da un indice spaziale; PostgreSQL usa l'indice GIST per escludere rapidamente le geometrie lontane.

Per ottenere distanza ordinata si può usare:

```
SELECT l.*, ST_Distance(
    l.coordinate::geography,
    ST_SetSRID(ST_MakePoint(:lon, :lat), 4326)::geography
) AS distance_m
FROM listing l
WHERE ST_DWithin(l.coordinate::geography, ST_SetSRID(ST_MakePoint(:lon,:lat),4326)::geography, :radius_meters
    )
ORDER BY distance_m ASC
LIMIT :limit OFFSET :offset;
```

### 2.4.6 La gestione delle immagini

La gestione delle immagini nel sistema segue il principio di separazione delle responsabilità: i contenuti binari (file immagine) sono memorizzati in uno storage oggetto scalabile (Amazon S3), mentre PostgreSQL conserva i metadati relazionali necessari per la gestione delle gallerie, l'ordinamento e i riferimenti agli annunci.

**Schema e metadati** La tabella dedicata `listing_image` contiene i campi principali necessari:

- `image_id` (PK)
- `created_at` (timestamp)
- `sort_order` (int) per stabilire l'ordine nella galleria
- `url` (varchar) che punta all'oggetto S3
- `listing_id` (FK verso `listing`)

Questa separazione consente operazioni transazionali sui metadati e sfrutta S3 per la consegna efficiente dei contenuti.

**Servizi e configurazione** La connettività con S3 è incapsulata in una classe di configurazione (vedi `S3Config.java`) e in un servizio applicativo che implementa l'interfaccia `ImageStorageService` (implementazione fornita da `S3ImageStorageService`). Il servizio si occupa di:

- Validare il file (MIME type, dimensione, magic bytes) tramite `isValidImage`.
- Generare chiavi oggetto sicure e prevedibili (es. `listings/listingId/uuid.ext`) con `buildObjectKey`.
- Caricare oggetti su S3 (`putObject`) e restituire URL/chiave.
- Eliminare singoli oggetti o tutti gli oggetti di un listing (prefisso), usando `listObjectsV2` + `deleteObjects` per cancellazioni bulk.
- Generare URL pre-firmati per accesso temporaneo (`generatePresignedUrl`) e fornire helper `presignUrl` per servire immagini private.

Questa integrazione fornisce un modello scalabile, sicuro e robusto per la gestione delle immagini, capace di mantenere la coerenza tra storage e metadati relazionali.

In questa sezione si documenta la struttura delle classi e dei moduli del backend. Il sistema è organizzato come un monolite modulare strutturato secondo principi DDD (**Domain-Driven Design**): ciascun package rappresenta un bounded context con responsabilità chiare ed è composto da:

- **Domain model**: le classi che codificano le entità di dominio (DAO/Entity).
- **Repository/DAO**: astrazioni per la persistenza (Spring Data Repository o implementazioni custom).
- **Service/Application**: logica applicativa, orchestrazione di repository ed integrazione esterna.
- **Controller / API layer**: punti di ingresso HTTP/REST.
- **DTO / Mapper**: oggetti di trasferimento e trasformazioni tra API e classi di dominio.

Questa separazione facilita testabilità, responsabilità chiare e limiti per l'evoluzione del sistema.

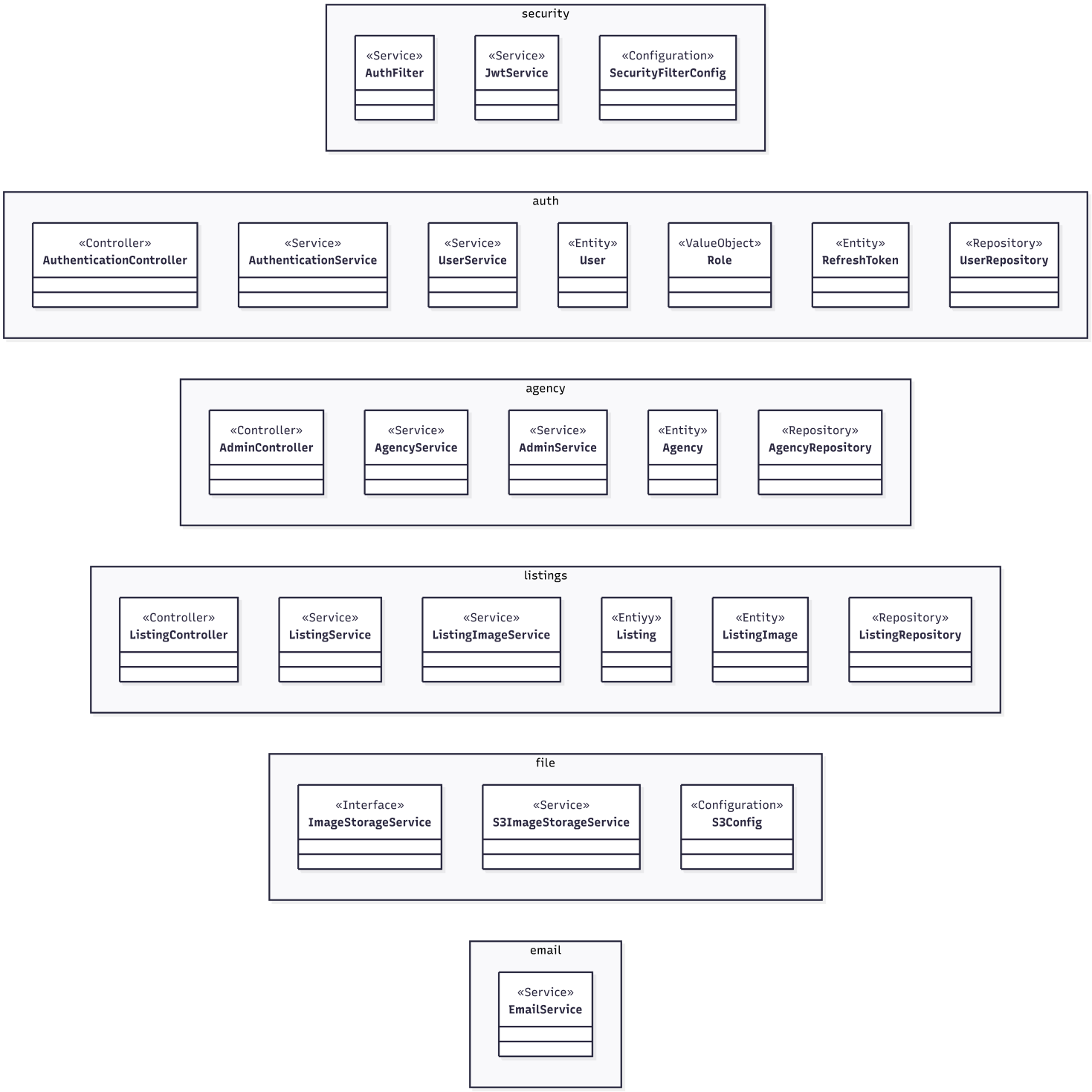


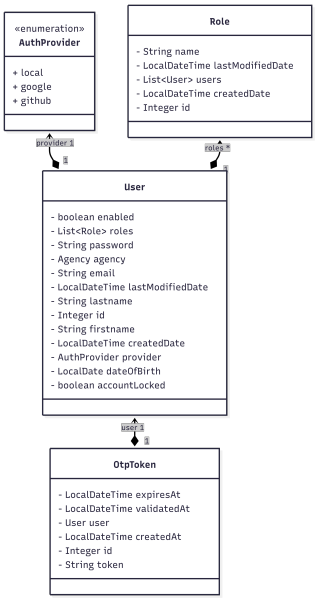

Figura 2.1: Package Diagram

### 2.5.1 Sommario dei package e classi principali

Di seguito le descrizioni per package principali presenti nel backend. Per ciascun package c'è un breve elenco delle classi di interesse (Controller, Service, Model, DTO, Repository).

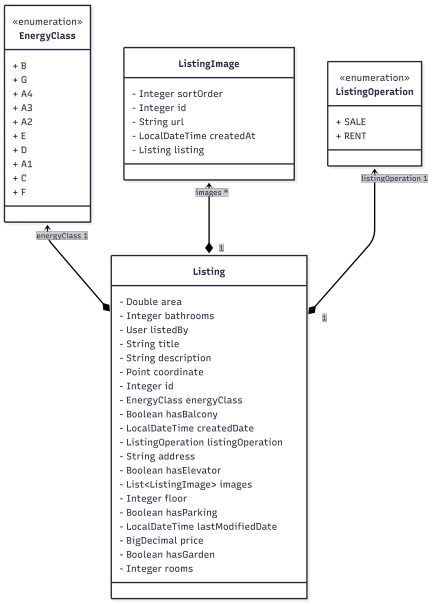
#### 2.5.1.1 Package `auth`

- Controller: `AuthenticationController`, `UserController` — espongono endpoint per login, registrazione, profilo, refresh token.
- Service: `AuthenticationService`, `UserService`, `RefreshTokenService`, `OtpService` — orchestrano autenticazione, gestione token e operazioni utente.
- Model: `User`, `Role`, `RefreshToken`, `OtpToken`.
- DTO:
  - `AuthenticationRequest`,
  - `AuthenticationResponse`,
  - `RegistrationRequest`,
  - `UpdateProfileRequest`,
  - `UserDetailsResponse`.
- Repository: `UserRepository`, `RoleRepository`, `RefreshTokenRepository`, `OtpTokenRepository`.



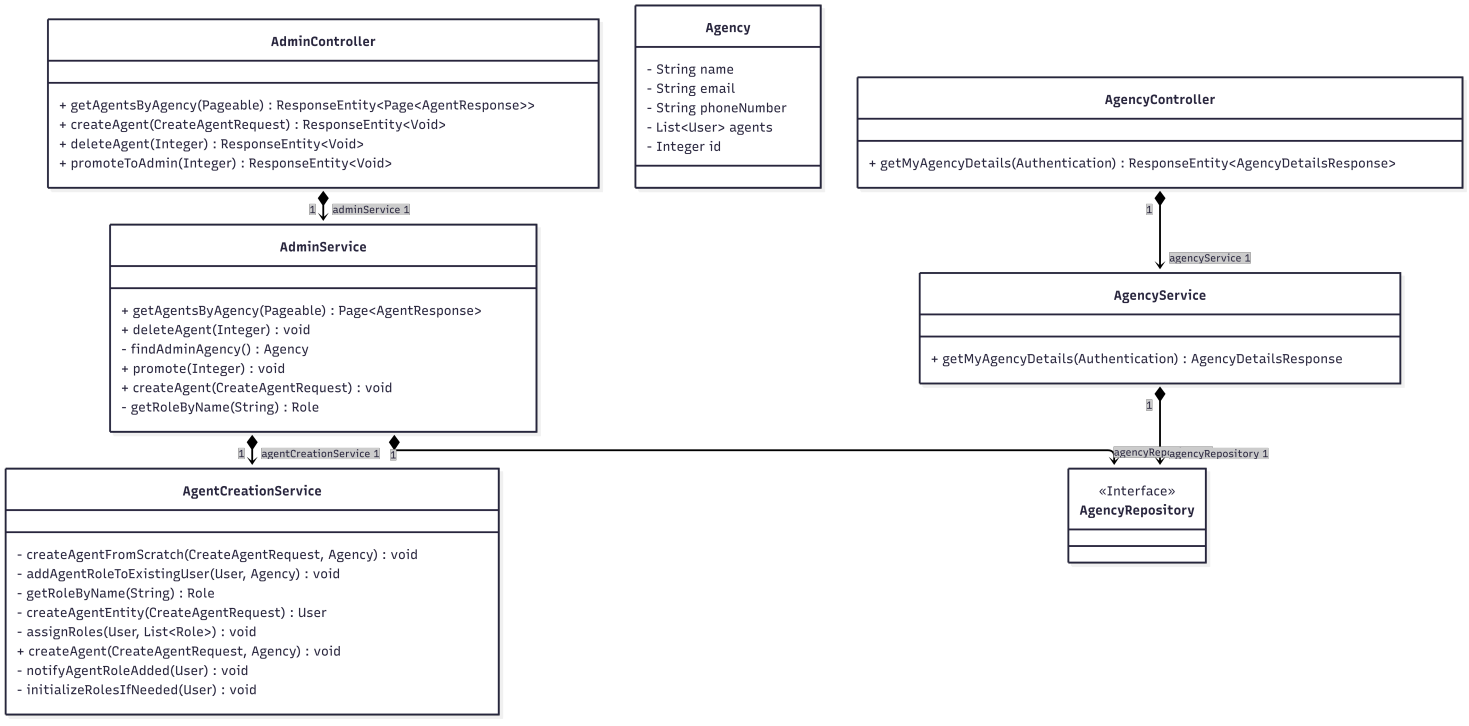

#### 2.5.1.2 Package `listings`

- Controller: `ListingController` — CRUD e ricerca annuncio.
- Service: `ListingService`, `GeometryService`, `ListingImageService` — logica di business, ricerche geospaziali (PostGIS) e gestione immagini metadata.
- Model: `Listing`, `ListingImage` — coordinate memorizzate come `geometry(Point,4326)`.
- DTO: request/response per create/update/list.
- Repository: `ListingRepository`, `ListingImageRepository`, `ListingSearchRepository` (implementazione custom per ricerche geospaziali ottimizzate).




#### 2.5.1.3 Package agency

- Controller: `AgencyController`, `AdminController` — gestione agenzie e operazioni amministrative sugli agenti.
- Service: `AgencyService`, `AdminService`, `AgentCreationService` — logiche di promozione agente e assegnazione agency.
- Model: `Agency`.
- Repository: `AgencyRepository`.




# 2.6 DIAGRAMMI DI SEQUENZA

### 2.6.1 Creazione annuncio immobiliare

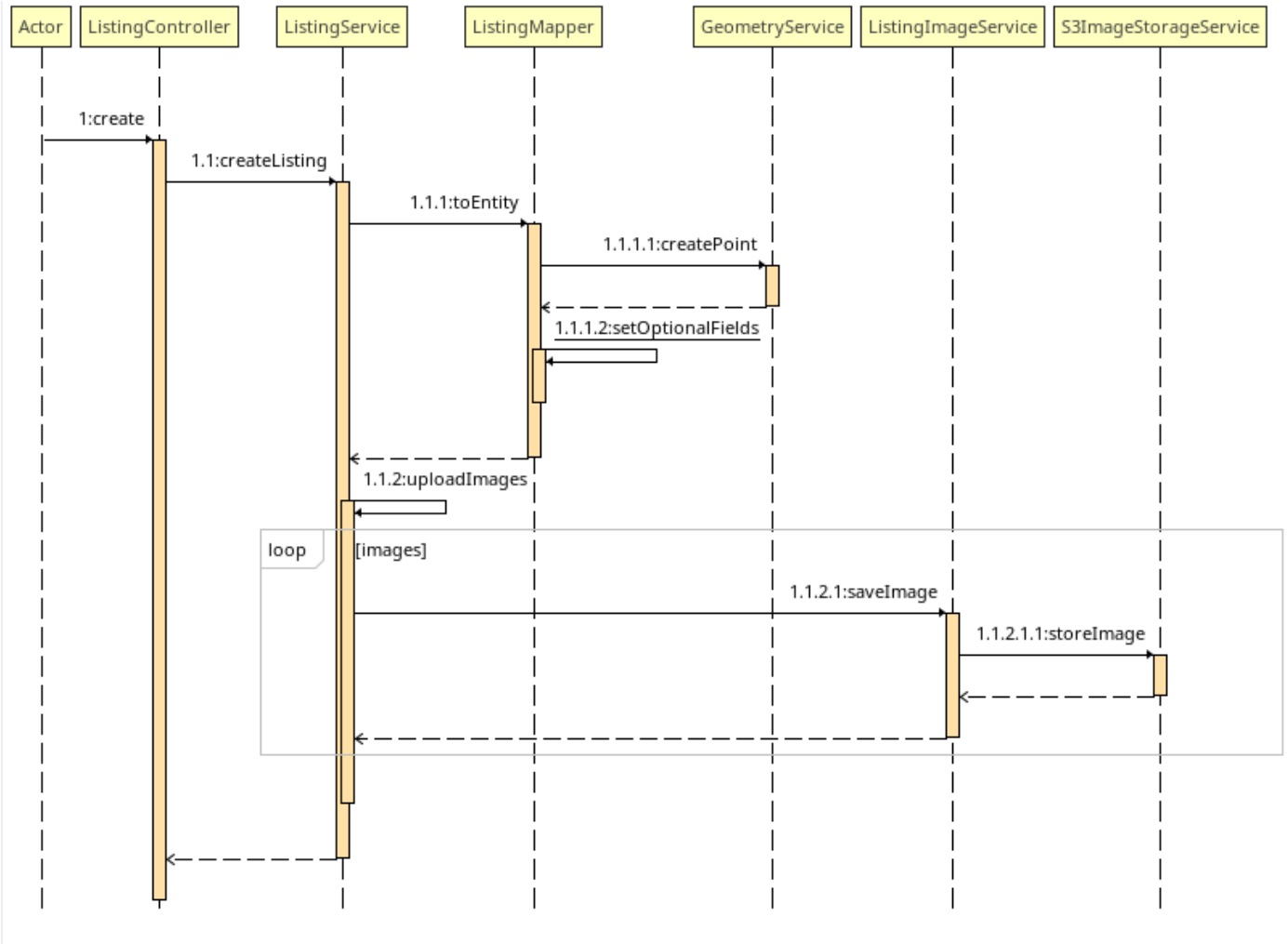



Figura 2.2: Sequence Diagram: `createListing`

### 2.6.2 Ricerca annuncio immobiliare

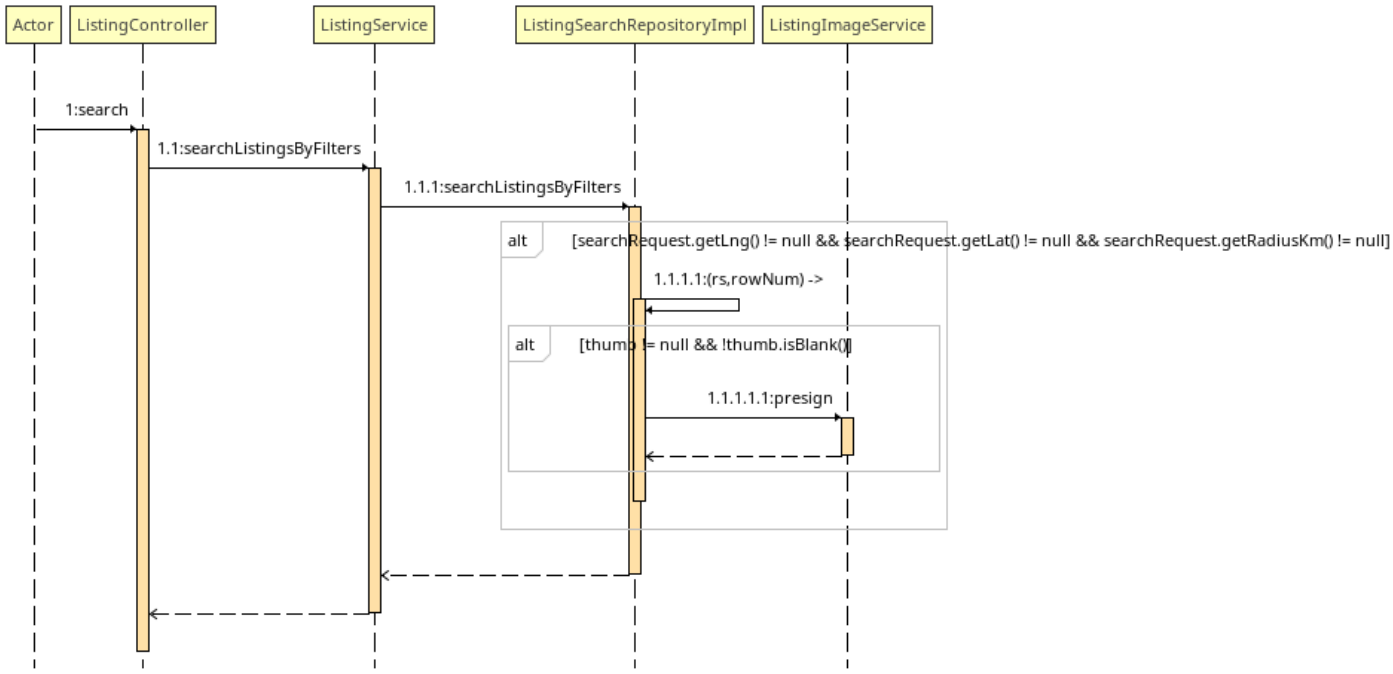



Figura 2.3: Sequence Diagram: `searchListing`

### 2.6.3 Creazione agente immobiliare

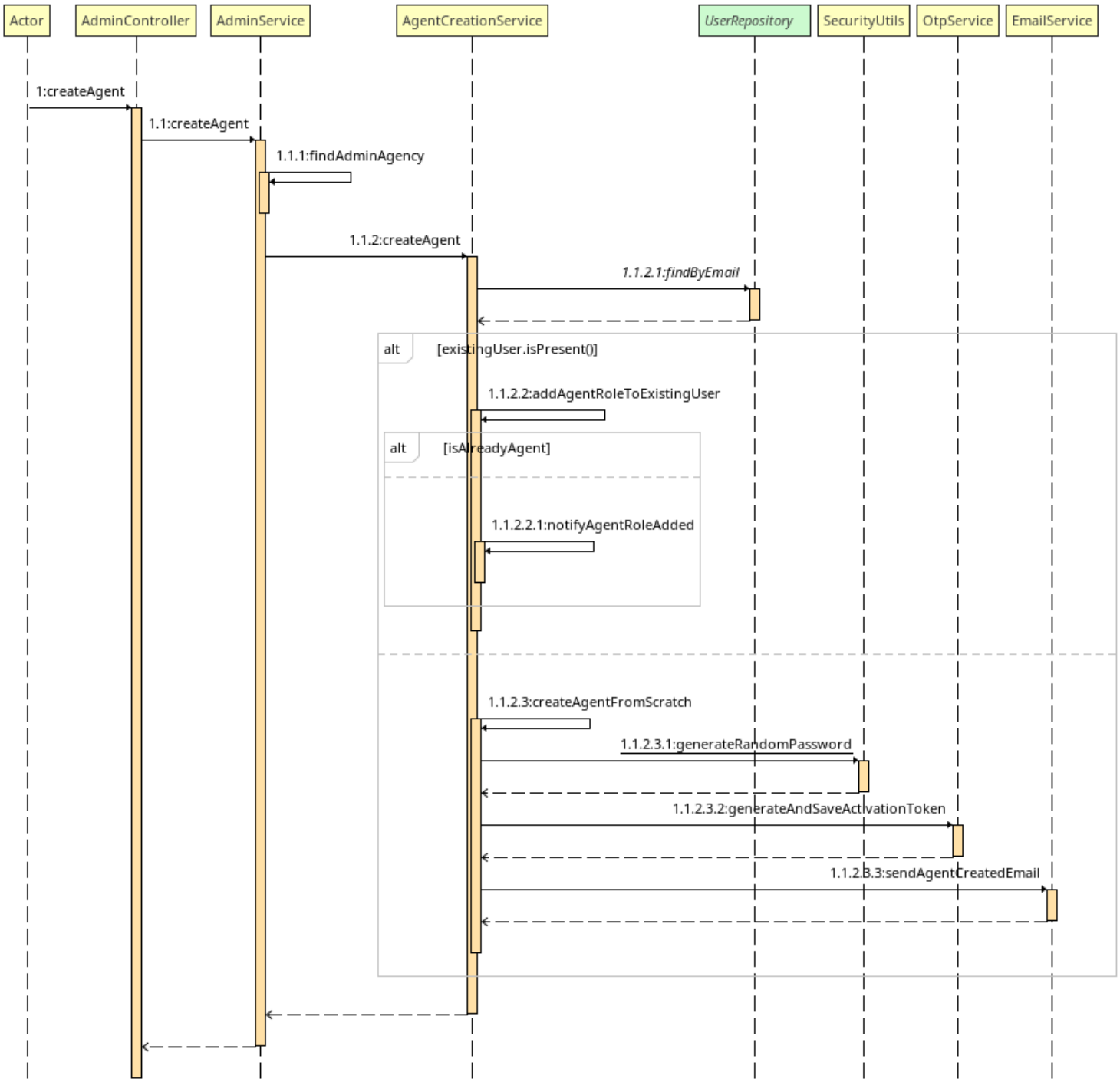



Figura 2.4: Sequence Diagram: `createAgent`

### 2.6.4 Promozione agente immobiliare

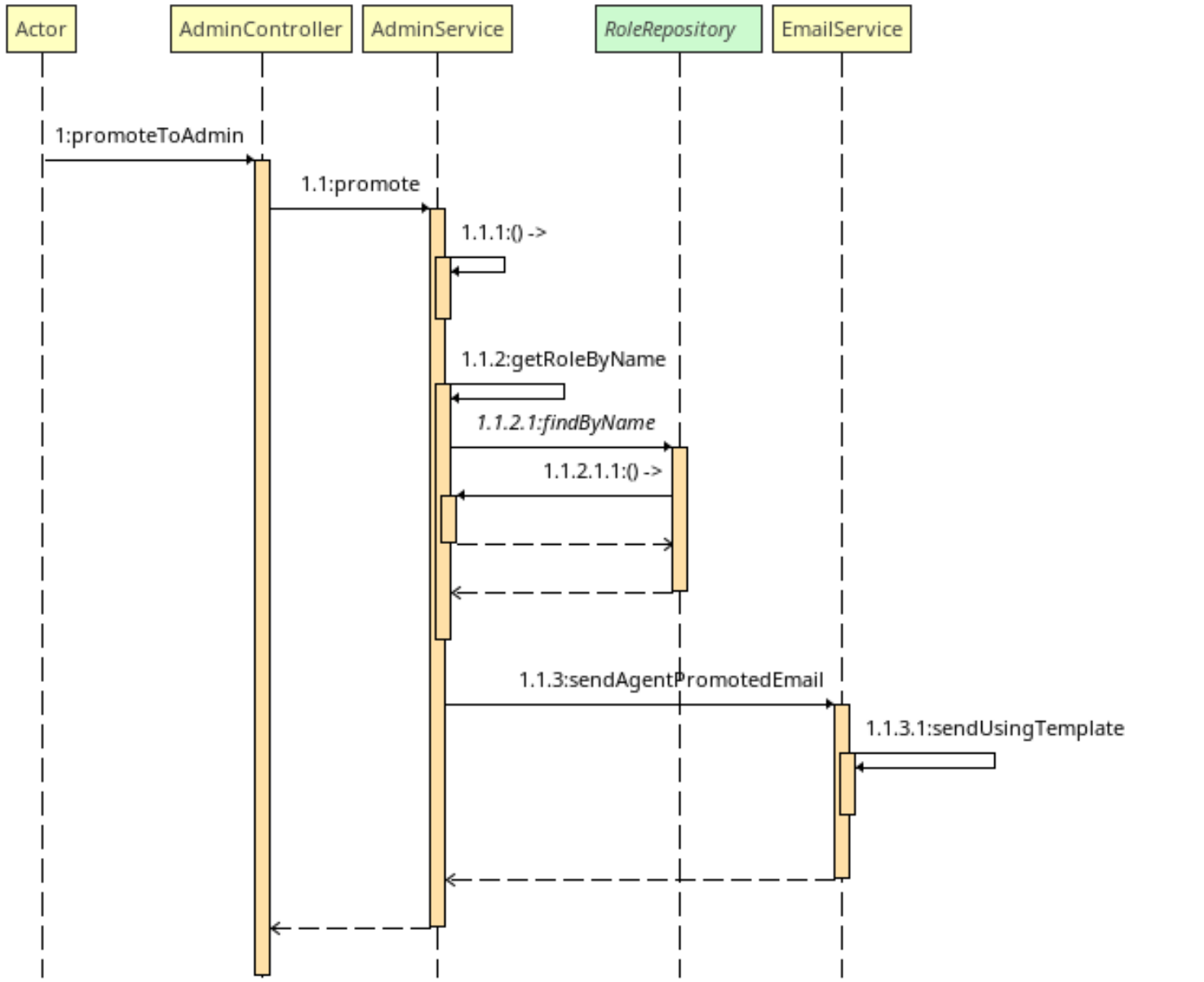



Figura 2.5: Sequence Diagram: `promoteAgent`

# Testing e valutazione dell'usabilità

## 3.1 Unit testing

### 3.1.1 Il metodo `updatePassword`

Il metodo `updatePassword` nella classe `UserService` è responsabile dell'aggiornamento della password di un utente. La logica prevede diversi controlli di validazione prima di procedere con la modifica: verifica che l'utente sia autenticato, che la vecchia password sia corretta, e che la nuova password sia valida e diversa da quella precedente.

Per garantire l'affidabilità e la correttezza di questo metodo, è stata progettata una suite di test di unità utilizzando il framework JUnit 5 e la libreria Mockito. Le strategie di test adottate sono una combinazione di tecniche di black-box e white-box.

#### 3.1.1.1 Strategie di Progettazione dei Test

**Black-Box Testing (Partizionamento in Classi di Equivalenza)** L'approccio black-box si concentra sulla verifica della funzionalità del metodo senza conoscere la sua implementazione interna. Abbiamo utilizzato la tecnica del partizionamento in classi di equivalenza per suddividere i dati di input in gruppi che dovrebbero essere elaborati allo stesso modo.

Sono state individuate le seguenti classi di equivalenza per gli input del metodo:

- **Utente corrente (`currentUser`):**
  - *Valida:* L'utente è un oggetto valido e non nullo.
  - *Non valida:* L'utente è `null` (simulando una richiesta non autenticata).
- **Esistenza dell'utente nel database:**
  - *Valida:* L'utente esiste nel repository.
  - *Non valida:* L'utente non viene trovato nel repository.
- **Vecchia password (`oldPassword`):**
  - *Valida:* La password corrisponde a quella memorizzata.
  - *Non valida:* La password non corrisponde.
- **Nuova password (`newPassword`):**
  - *Valida:* Stringa non vuota, non blank e diversa dalla vecchia password.
  - *Non valida (blank):* Stringa vuota, contenente solo spazi, o nulla.
  - *Non valida (identica):* Uguale alla vecchia password.

I test sono stati creati per coprire almeno un rappresentante di ciascuna classe di equivalenza, sia valida che non valida, per assicurare che il sistema gestisca correttamente tutti gli scenari di input.

**White-Box Testing (Copertura Strutturale)** L'approccio white-box si basa sulla struttura interna del codice per derivare i casi di test. L'obiettivo è stato quello di ottenere una copertura elevata delle diramazioni (branch coverage), assicurando che ogni percorso decisionale all'interno del metodo fosse testato.

Ogni blocco condizionale (`if`) nel metodo `updatePassword` è stato coperto da uno specifico caso di test:

- Caso in cui l'utente è `null` è testato da: **updatePassword_currentUserIsNull_throwsBadCredentialsException**.
- Il ramo in cui l'utente non viene trovato è testato da: **updatePassword_userNotFoundInRepository_throwsUsernameNotFoundException**.
- Il ramo in cui la vecchia password è errata è coperto da: **updatePassword_oldPasswordIncorrect_throwsBadCredentialsException**.
- La condizione di password blank è testata da: **updatePassword_newPasswordIsBlank_throwsBadRequestException**.
- Il caso in cui la nuova password è uguale alla vecchia è testato da: **updatePassword_newPasswordIsSame_throwsBadRequestException**.
- Il caso in cui tutti i controlli passano è verificato da: **updatePassword_success**.

### 3.1.2 Il metodo `createPoint`

Il metodo `createPoint` in `GeometryService` ha il compito di creare un oggetto geometrico `Point` a partire da coordinate di longitudine e latitudine. La sua responsabilità principale è validare che le coordinate di input siano numeriche e finite prima di delegare la creazione del punto alla libreria JTS (Java Topology Suite).

La progettazione dei test di unità per questo metodo si è basata su diverse tecniche per garantirne la robustezza e la correttezza.

#### 3.1.2.1 Strategie di Progettazione dei Test

**Black-Box Testing (Partizionamento in Classi di Equivalenza e Analisi dei Valori Limite)** Le coordinate (longitudine e latitudine) sono state analizzate per identificare classi di input e valori limite significativi.

- **Classi di Equivalenza:**
  - *Numeri finiti validi:* Include valori positivi, negativi e zero. Questi input dovrebbero portare alla creazione con successo di un oggetto `Point`. I test:
    - **createPoint_ValidCoordinates**,
    - **createPoint_NegativeCoordinates**,
    - **createPoint_ZeroCoordinates**

    coprono questa classe.
  - *Valori speciali non finiti:* Include `Double.NaN`, `Double.POSITIVE_INFINITY`, e `Double.NEGATIVE_INFINITY`. Questi input dovrebbero sollevare un'eccezione, come verificato in:
    - **createPoint_SpecialDoubleValues_ThrowsException**
- **Analisi dei Valori Limite:**
  - Sono stati testati i valori limite convenzionali per le coordinate geografiche (es. -180, 90) in
    - **createPoint_BoundaryCoordinates**

    per assicurare che vengano gestiti correttamente.
  - È stato incluso anche un test per valori finiti ma fuori dal range standard (es. 200.0) in
    - **createPoint_OutOfRangeCoordinates**

    per confermare il comportamento atteso (la libreria JTS crea il punto senza sollevare eccezioni).

**White-Box Testing (Copertura delle Condizioni)** L'analisi white-box si è concentrata sulla struttura interna del metodo. Esso contiene un'unica istruzione `if` con una condizione composta da quattro parti:

```
if (Double.isNaN(lng) || Double.isInfinite(lng) ||
    Double.isNaN(lat) || Double.isInfinite(lat)) { ... }
```

L'obiettivo è stato ottenere la **copertura delle condizioni (Condition Coverage)**, assicurando che ogni singola sotto-condizione venisse valutata sia a `true` che a `false`.

- Il percorso in cui la condizione è `false` (input valido) è coperto dalla maggior parte dei test, che verificano la creazione del punto.
- Il percorso in cui la condizione è `true` (input non valido) è coperto dal test:
  - **createPoint_SpecialDoubleValues_ThrowsException**.

### 3.1.3 Il metodo `saveImage`

#### 3.1.3.1 Strategie di Progettazione dei Test

La progettazione dei test di unità per il metodo `saveImage` della classe `ListingImageService` si è focalizzata sulla verifica delle interazioni con i servizi esterni (storage di file e database) e sulla corretta gestione della logica di business per l'ordinamento delle immagini. Sono state applicate principalmente tecniche di black-box, simulate attraverso l'uso di mock.

**Black-Box Testing (Partizionamento in Classi di Equivalenza)** Sono state identificate le seguenti classi di equivalenza basate sugli input, sullo stato del sistema e sugli esiti delle dipendenze:

- **Argomenti di input (`file`, `listing`):**
  - *Validi:* Gli argomenti sono oggetti non nulli.
  - *Non validi (file nullo):* L'argomento `file` è `null`. Questo caso è coperto da:
    - **saveImage_nullFile_throwsException**
  - *Non validi (listing nullo):* L'argomento `listing` è `null`. Questo caso è coperto da:
    - **saveImage_nullListing_throwsException**
- **Stato dell'annuncio (Listing):**
  - *Annuncio senza immagini esistenti:* L'operazione di salvataggio deve assegnare all'immagine un `sortOrder` pari a 1. Questo scenario è coperto dal test:
    - **saveImage_firstImageForListing_success**
  - *Annuncio con immagini esistenti:* Il `sortOrder` della nuova immagine deve essere calcolato come il massimo valore esistente incrementato di uno. Questo è verificato dal test:
    - **saveImage_withExistingImages_success**
- **Esito del caricamento del file su S3:**
  - *Successo:* Il servizio di storage (`S3ImageStorageService`) restituisce un URL valido. Questo è il caso base per i test di successo.
  - *Fallimento:* Il servizio di storage solleva un'eccezione. Questo scenario è coperto da:
    - **saveImage_s3UploadFails_throwsException**

    che verifica la corretta propagazione dell'errore.

**White-Box Testing (Copertura Strutturale)** L'approccio white-box garantisce che i principali percorsi logici ed eccezionali del metodo siano testati.

- **Copertura delle Decisioni:** La principale struttura condizionale, `(maxSortOrder != null)`, è coperta nei suoi esiti `true` e `false` rispettivamente da **saveImage_withExistingImages_success** e **saveImage_firstImageForListing_success**.
- **Gestione delle Eccezioni:** I test assicurano che i percorsi di fallimento vengano gestiti correttamente, prevenendo l'esecuzione di codice che altererebbe lo stato del sistema in modo inconsistente.
  - **saveImage_s3UploadFails_throwsException**: verifica che un fallimento nello storage non porti a operazioni sul database.
  - **saveImage_nullListing_throwsException**: verifica che un input nullo per il listing causi un'eccezione immediata (`NullPointerException`) prima di qualsiasi interazione con i servizi esterni.
  - **saveImage_nullFile_throwsException**: verifica che la chiamata con un file nullo propaghi l'eccezione sollevata dal servizio di storage.

### 3.1.4 Il metodo `createListing`

#### 3.1.4.1 Strategie di Progettazione dei Test

Il metodo `createListing` nella classe `ListingService` gestisce la logica di creazione di un nuovo annuncio immobiliare. Le sue responsabilità includono la validazione del contesto di sicurezza (l'utente deve essere un agente autenticato), la mappatura dei dati di input in un'entità, il salvataggio dell'entità nel database e il caricamento delle immagini associate.

La strategia di test combina un approccio black-box per validare gli scenari di input e di stato del sistema, e un approccio white-box per garantire che i percorsi critici e le gestioni delle eccezioni siano coperti.

**Black-Box Testing (Partizionamento in Classi di Equivalenza)** Sono state identificate le seguenti classi di equivalenza per gli input e lo stato del sistema:

- **Stato di Autenticazione dell'Utente:**
  - *Utente autenticato e presente nel database:* Questo è lo scenario di successo, coperto dal test
    - **createListing_validRequestAndImages_success**.
  - *Utente autenticato ma non trovato nel database:* Questo input non valido dovrebbe impedire la creazione dell'annuncio. È coperto da **createListing_authenticatedUserNotFound_throwsException**, che verifica il lancio di `IllegalStateException`.
- **Lista delle Immagini (`images`):**
  - *Lista valida e non vuota:* Caso di successo coperto da **createListing_validRequestAndImages_success**.
  - *Lista nulla:* Input non valido coperto da **createListing_nullImages_throwsException**, che si aspetta un'`IllegalArgumentE`
  - *Lista vuota:* Input non valido coperto da **createListing_emptyImages_throwsException**, che si aspetta anch'esso un'`IllegalArgumentException`.
- **Esito del Salvataggio delle Immagini:**
  - *Successo:* Tutte le immagini vengono salvate correttamente dal `ListingImageService`.
  - *Fallimento:* Il salvataggio di un'immagine fallisce, sollevando un'eccezione. Questo scenario, coperto da:
    - **createListing_imageUploadFails_shouldRollback**

    è cruciale per testare la natura transazionale del metodo. Il test verifica che l'eccezione si propaghi, implicando un rollback della creazione dell'annuncio.

**White-Box Testing (Copertura Strutturale)** L'analisi white-box si concentra sulla copertura dei percorsi decisionali e di gestione degli errori all'interno del metodo e dei suoi collaboratori privati.

- **Copertura dei Rami (Branch Coverage):** I test coprono i rami principali. Ad esempio, la logica nel metodo privato `getAgent` viene testata per i casi in cui l'utente viene trovato o meno. Il metodo `uploadImages` viene testato per i casi di lista nulla, vuota o valida.
- **Gestione delle Eccezioni:** La suite di test verifica che il sistema si comporti correttamente in caso di fallimento. I test si assicurano che le eccezioni vengano sollevate e propagate come atteso e che non vengano eseguite operazioni di persistenza parziali (es. creazione dell'annuncio senza che l'utente esista).

## 3.2 Tecniche di valutazione dell'usabilità

### 3.2.1 Expert reviews

### 3.2.2 Esperimenti con utenti reali

#### 3.2.2.1 Soggetti reclutati, procedura sperimentale e metriche considerate

#### 3.2.2.2 Survey sottoposto agli utenti

#### 3.2.2.3 Risultati dell'esperimento